

Introduction to Sphere Tracing

Final Year Project

1811473

A dissertation presented for the degree of
Mathematics with Computer Science

Supervisor: Alexey Pichugin

Department of Mathematics

Brunel University London

2021/2022

Acknowledgements

It would not be possible to complete this paper if it weren't for the help of my supervisor, Alexey Pichugin. His support and encouragement has both made this work fulfilling and enjoyable, and has also greatly increased my interest in computer graphics.

In addition, I would also like to thank my friends at Brunel for their support not only for this dissertation but for my entire time at Brunel.

Abstract

In this paper, we will be implementing the rendering technique known as sphere tracing in the field of computer graphics, but taking a more mathematical approach instead of a computer science one.

We will start with the definition of ray tracing and how the limitations of mathematically calculating an intersection hinders the complexity of geometry that can be rendered by this technique. This is where ray marching comes in, as an approximation to solve the issues that calculating an intersection presented. Sphere tracing is a method of ray marching.

Sphere tracing relies on the use of signed distance functions. These are essentially functions of geometry which return the shortest distance from a point to the surface of the geometry. This is how we will be defining shapes in the computer graphic scene.

It is not enough that we define the shapes in the scene, as without lighting the scene the objects will not be visible. This paper will therefore also discuss the Lambertian lighting model alongside other contributions to a more realistic scene.

This will all be compiled into Shadertoy, a web-driven IDE which uses the language GLSL. The paper will seek to implement the language to generate the scenery within Shadertoy.

Contents

1	Introduction	5
1.1	Introduction to Computer Graphics	5
1.2	Classical Ray Tracing	6
1.2.1	Introduction to Ray Tracing	6
1.2.2	Computing an intersection for ray tracing	7
1.3	Ray Marching and Sphere Tracing	9
2	Signed Distant Functions with generating relevant geometry	11
2.1	Signed Distance Functions (SDFs) and Signed Distance Bounds (SDBs) . .	11
2.2	Finding SDBs with Lipschitz bounded functions	12
2.3	Constructive Solid Geometry	14
2.4	A simple scene with a plane and a sphere	15
2.5	Signed Distant Function of a capsule	16
2.6	Signed Distant Function of a torus	17
2.7	Signed Distant Function of a cuboid	18
2.8	Signed Distant Function of a cylinder	19
2.8.1	Interior distance	20
3	Illumination and Shading theory	21
3.1	Introduction	21
3.2	Lighting	21
3.2.1	Directional light	21
3.2.2	Point light	21
3.2.3	Lambertian reflection	21
3.3	Shadows	24
3.4	Materials	24
3.5	Reflections	24
3.5.1	Recursive reflections	24
3.5.2	Specular reflection (Phong illumination model)	25
3.5.3	Fresnel term	26
4	Implementation with GLSL	27
4.1	Introduction to GLSL and Shadertoy	27
4.2	Ray Origin and Ray Direction	27
4.3	The sphere tracing function	28
4.4	Implementation of SDFs	30
4.4.1	Sphere on ground plane	30
4.4.2	Capsule	31
4.4.3	Torus	32
4.4.4	Cuboid	33
4.4.5	Cylinder	34
4.4.6	Showing the geometry within shadertoy	35
4.5	Implementation of shadows	35
4.6	Implementation of materials	37
4.7	Implementation of Phong's illumination	39
4.8	Implementation of reflection	40
4.9	Implementation of the Fresnel term	42

4.10	The final picture	43
5	Conclusion	45
5.1	Summary	45
5.2	Limitations	45
5.2.1	Chapter 1: Introduction	45
5.2.2	Chapter 2: Signed distant functions with generating relevant geometry	46
5.2.3	Chapter 3: Illumination and shading theory	46
5.2.4	Chapter 4: Implementation with GLSL	46
5.3	Appraisals	46
5.4	Further study	47
6	Appendix	48
7	Bibliography	59

1 Introduction

1.1 Introduction to Computer Graphics

Computer graphics has existed since the early 1940s [1], originally purposed for engineers and scientists, and only started becoming what we understand today as computer graphics in the 1980s with the release of commercial personal computers. These computers started with the Xerox Star and later was beaten by the more competitively priced IBM and Apple computers (something we don't see often with Apple these days!).

The technology itself serves as a method of displaying scenes on a computer screen as a two-dimensional image. Whilst starting with just displaying data and figures on a screen, in the present day we are capable of displaying very complex 3D geometry, and controlling the geometry with the input devices we have at our disposal. Popular applications of these complex geometries using input devices include video games, which have evolved and used many techniques over the decades to produce amazingly stylised, or perhaps even realistic computer graphics.

Put simply, computer graphics are a core technology in a large number of industries, functioning in practically every computer screen we see today.

Within the study of computer graphics, there exists methods of determining the visibility of objects in the space by firing lines, formally called rays, into a coordinate space (x, y, z) . In that space also exists the virtual observer, the virtual screen and the generated scene. A name for these methods is called ray tracing, first developed by Arthur Appel in 1968 [2, pg.701].

The task we aim to accomplish is, using a method of generating computer graphics, producing a virtual screen that should contain an image of the three-dimensional scene from the point of view of the virtual observer.

1.2 Classical Ray Tracing

1.2.1 Introduction to Ray Tracing

Ray tracing is when rays of light are pursued from the virtual observer to the 3D object. We have: the virtual observer which is a point in the coordinate space which represents where the viewer of the screen will see the object from, a virtual screen which is divided into a regular grid as shown in figure 1, and a number of objects in a space. For every pixel within the screen, a ray or multiple rays are fired from the virtual observer, through the screen and into the space where it will either intersect with an object or hit a wall within the scene. The colour (and intensity of the colour) of the pixel on the screen will correspond with the colour of the intersected object or wall. The resulting screen is the ray traced image.

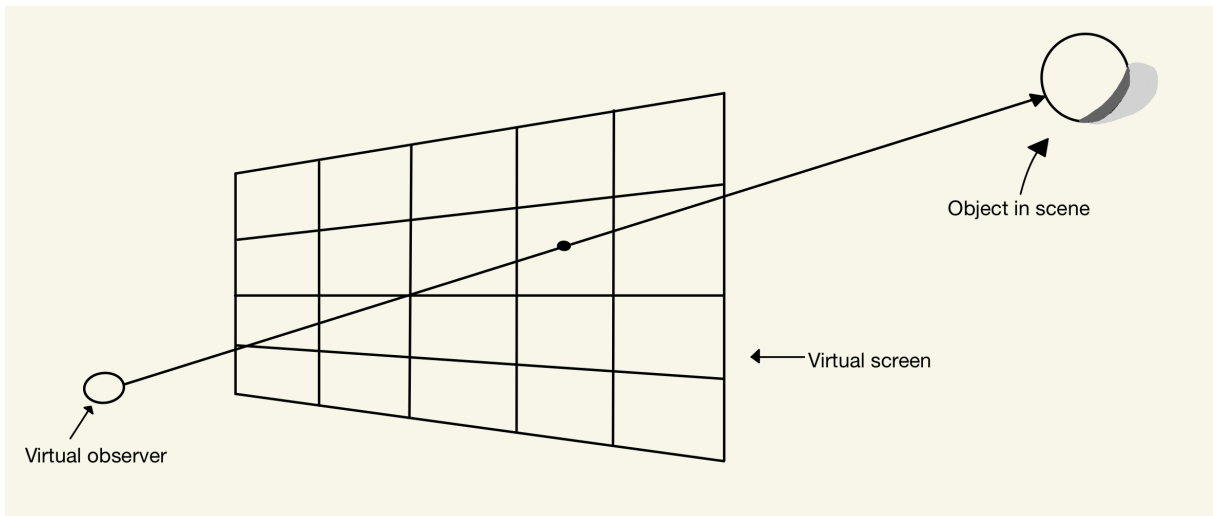


Figure 1: Image showing a ray fired from the virtual observer through a pixel on the screen hitting an object on the scene.

In the figure above, we see the virtual observer firing a single ray at the object in scene which is a shaded sphere. That ray maps the point of the sphere where the ray intersects with the sphere as a pixel on the screen which the ray passes through. With multiple rays firing out, we can get a ray traced image generated on the virtual screen.

1.2.2 Computing an intersection for ray tracing

For any ray tracer, it is imperative to be able to calculate the intersections of the ray and any objects. We can represent the ray as a line, using parametric representations.

As written in [2, pg.702], we can define each (x, y, z) point along the ray from the virtual observer, also written as (x_0, y_0, z_0) , to the screen, written as (x_1, y_1, z_1) using a value t such that,

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix} + t \begin{pmatrix} x_1 - x_0 \\ y_1 - y_0 \\ z_1 - z_0 \end{pmatrix} \quad (1.1)$$

In parametric form, this is:

$$x = x_0 + t(x_1 - x_0) \quad y = y_0 + t(y_1 - y_0) \quad z = z_0 + t(z_1 - z_0)$$

We will be using these equations above, and so for convenience we will define the following:

$$\Delta x = x_1 - x_0 \quad (1.2)$$

$$\Delta y = y_1 - y_0 \quad (1.3)$$

$$\Delta z = z_1 - z_0 \quad (1.4)$$

Any distance between the virtual observer and the screen, the value of t will be between 0 and 1. Any negative values of t means that the point is behind the virtual observer, and finally any values of $t > 1$ means the point is beyond the screen. Mathematically we can determine the value of t by expanding the equation of the object in the space and substituting in the values for x, y and z in, and this is the principle of how Appel's ray tracer works.

Appel applies this to the implicit equation of a sphere, written below, and solves to get a quadratic in terms of t . If there are no real roots to the equation, then the ray and the sphere do not intersect. One root means that the ray is a tangent to the sphere and two roots means that there is an intersection. The root with the lowest positive t value is the one closest to the screen. This is simplified by normalising the ray such that the distance between the screen and the virtual observer to 1. The calculation will then give a coefficient of 1 for t^2 which is then easier to solve.

$$x^2 + y^2 + z^2 - r^2 = 0 \quad (1.5)$$

Computing t for a sphere (with origin at (a, b, c)) is relatively straightforward, as it only involves solving a quadratic equation.

The intersection is then found by expanding the equation and substituting in the values of x, y , and z we discussed above.

$$x^2 - 2ax + a^2 + y^2 - 2by + b^2 + z^2 - 2cz + c^2 = r^2 \quad (1.6)$$

$$\begin{aligned} & (x_0 + t\Delta x)^2 - 2a(x_0 + t\Delta x) + a^2 \\ & + (y_0 + t\Delta y)^2 - 2b(y_0 + t\Delta y) + b^2 \\ & + (z_0 + t\Delta z)^2 - 2c(z_0 + t\Delta z) + c^2 \\ & = r^2 \end{aligned}$$

$$\begin{aligned}
& x_0^2 + 2x_0\Delta x t + \Delta x^2 t^2 - 2ax_0 - 2a\Delta x t + a^2 \\
& + y_0^2 + 2y_0\Delta y t + \Delta y^2 t^2 - 2by_0 - 2b\Delta y t + b^2 \\
& + z_0^2 + 2z_0\Delta z t + \Delta z^2 t^2 - 2cz_0 - 2c\Delta z t + c^2 \\
& = r^2
\end{aligned}$$

Collecting the like terms gives:

$$\begin{aligned}
& (\Delta x^2 + \Delta y^2 + \Delta z^2)t^2 + 2t(\Delta x(x_0 - a) + \Delta y(y_0 - b) + \Delta z(z_0 - c)) \\
& + (x_0^2 - 2ax_0 + a^2 + y_0^2 - 2by_0 + b^2 + z_0^2 - 2cz_0 + c^2) \\
& - r^2 = 0
\end{aligned}$$

$$\begin{aligned}
& (\Delta x^2 + \Delta y^2 + \Delta z^2)t^2 + 2t(\Delta x(x_0 - a) + \Delta y(y_0 - b) + \Delta z(z_0 - c)) \\
& + (x_0 - a)^2 + (y_0 - b)^2 + (z_0 - c)^2 - r^2 = 0
\end{aligned}$$

The resulting equation gives us a quadratic in t , which can be solved using the quadratic formula to find the intersection point(s) of the ray and the sphere.

The problem with Appel's ray tracing method is when there are more intersections with the shape, such as with a torus. A torus when intersected with a ray will result in a quartic equation which is far more difficult to solve. In figure 2, we have a visual representation of the intersection with a torus. Furthermore, multiple intersections with different objects is possible. A scene involving 5 spheres and 5 tori would result in having to compute 5 quadratic and 5 quartic equations. The computation complexity quickly increases!

Deriving equations for more complex objects is rarely practical. Often as a result of this, approximations for the object surfaces are made using triangular meshes. Though this technique is not a trick solution to the problem at hand, as an intersection with a triangle involves solving a system of simultaneous linear equations [4]. Further complications arise when we take note that the first intersection is not sufficient to determine the colour of the pixel on the screen. As an example, calculating the shadows of an object requires checking the lines of visibility between the object and every light source. To calculate reflections, one needs to implement recursive algorithms which follow additional rays from the intersection point, first introduced by Whitted [2, pg.778]. Recursion makes the calculations near-impossible to calculate in real time.

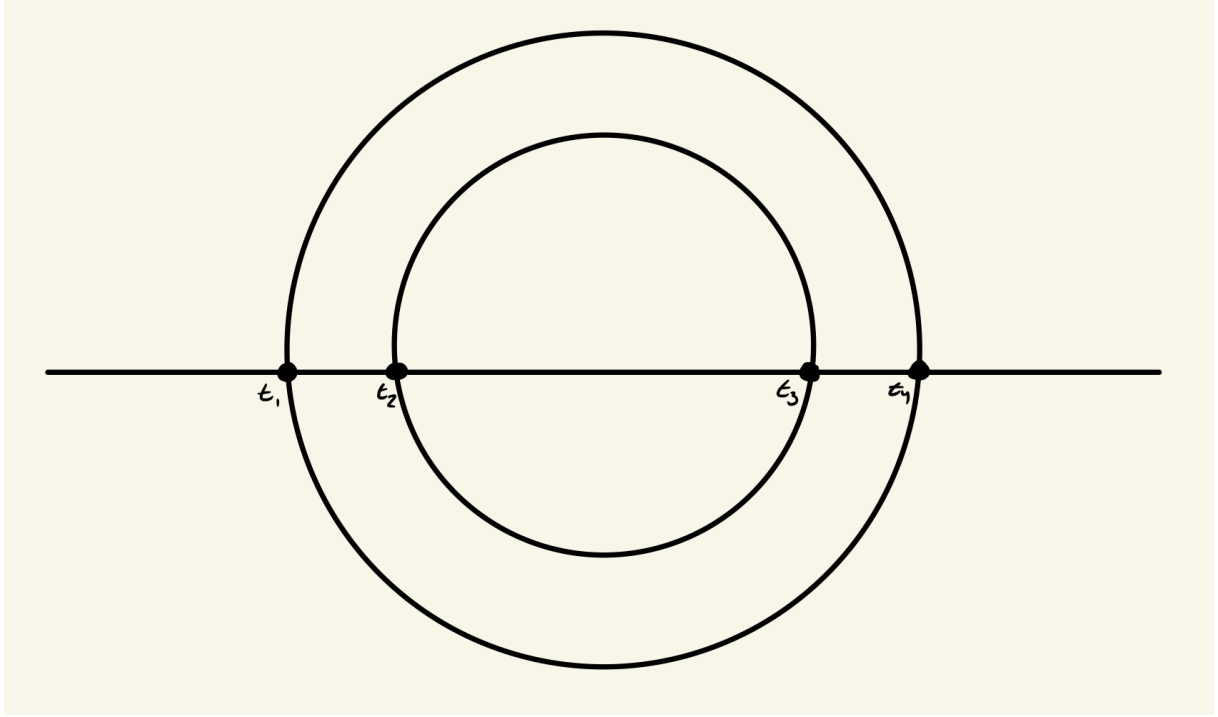


Figure 2: Image showing a two-dimensional torus intersected by a ray

1.3 Ray Marching and Sphere Tracing

Ray tracing as a foundation was very helpful in developing computer graphics. However, due to the increasing difficulties working with complex three-dimensional geometry, we seek other methods of generating computer graphics efficiently.

Ray marching is the idea that instead of calculating the ray intersection through substitution, you take an iterative approach where you take steps up and/or down the ray until you find the intersection. The intersection is formally defined as the point where the line of the ray meet with a surface. There are many types of ray marching, such as ray casting or bisection ray marching, however the approach this project will encompass is Sphere Tracing, first introduced by J. Hart [3].

Sphere tracing, as the name suggests, generates spheres along the ray with a radius from the line equal to (or less than as we will discuss later) the distance to the closest object. At the beginning of the ray, we find the closest object to the ray from that point and we create a sphere with the radius equal to the distance between that point and the object. We follow that radius length along the ray and repeat at that point until the radius is so small that we say we have an intersection between the ray and an object. In figure 3, we see a two dimensional representation of this using circles instead of spheres.

We can see from the diagram that the circle essentially gives an area where there are no intersections with any object. There needs to be some conditions in place to stop the march. A stop would occur for one of two reasons, we find an intersection with an object or if we have travelled too far. Additionally, we might only wish to do a certain number of steps (N steps) along the ray, in which case we would stop after hitting the maximum allowed number of steps.

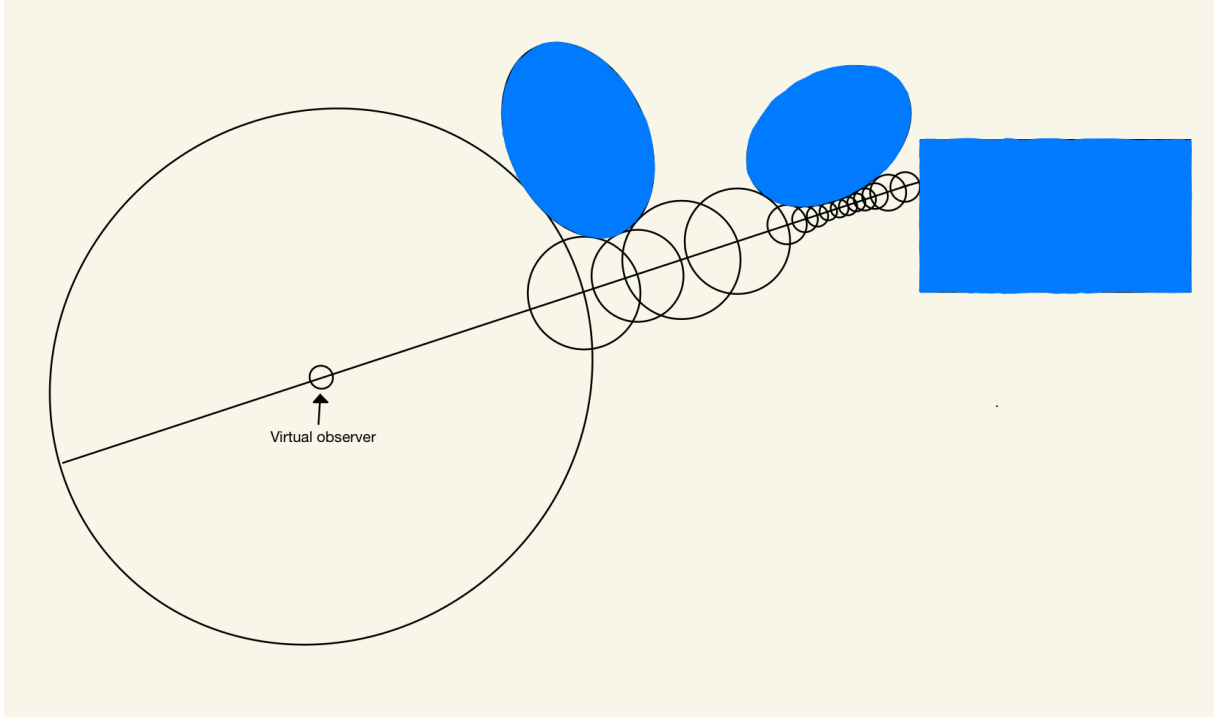


Figure 3: Image showing circle tracing

Algorithm 1 Simple Sphere Tracer Pseudocode

1: Initialise $t = 0$

2: **while** $t \leq D$ **do**

Ensure: $d = f(r(t))$

3: **if** $d \leq \epsilon$ **then** *return* t

▷ INTERSECTION

4: **end if**

5: $t = t + d$

6: **end while**

7: *return* θ

▷ NO INTERSECTION

The pseudocode above adapted from Hart’s paper [3, pg.531] serves to show how implicit surfaces are rendered. The signed distant bound, f , ray, $r(t)$, and maximum distance the ray tranversed, D , are used. The code essentially takes steps along the ray equal to size d , and if the next step size is smaller than a threshold value ϵ then we consider an intersection to be found. Otherwise there is no intersection. Our approximation to find the intersection will be similar to this with a few improvements.

This method fundamentally relies upon the possibility of computing the shortest distance from a point on a ray to the implicit surface of every object in the scene. This may seem incredibly complex computationally, however there is a very efficient method for a wide variety of shapes stretching further than what classical ray tracing is capable of.

2 Signed Distant Functions with generating relevant geometry

2.1 Signed Distance Functions (SDFs) and Signed Distance Bounds (SDBs)

A signed distant function returns the distance from the solid geometry to a point in space. A convention commonly used is that the sign of the distance is negative when the distance ends inside the object and positive outside ($f(x) < 0$ inside the object, $f(x) \geq 0$ outside the object). SDFs specifically return the exact distance to the intersection, and the function used in the SDF is different for every shape. Calculating the distance to the cube would be different for example than calculating the distance to a sphere.

We've been mentioning in the previous section implicit surfaces. Implicit equations are defined as:

$$F(x, y, z) = 0 \quad (2.1)$$

Implicit surfaces are functions where they aren't solved for a specific variable, (x, y, z) in this case. For the function to be explicit, it would have to be equated to either of the variables, for example $z = f(x, y)$. Sphere tracing is used to determine whether the point is inside or outside the implicit surface.

Suppose that we have a geometric object A implicitly defined by the continuous function $f(x)$ with the convention described above regarding the sign of the distance. With this convention, you can say that the multivalued function $f^{-1}(0)$ is the surface of the object. Performing an inversion of the implicit function would be impossible in most cases, however it provides us with a convenient mathematical notation for the surface of the shape.

We call the function, $f(x)$, a signed distance function **if and only if**:

$$f(x) = d(x, f^{-1}(0)), \quad (2.2)$$

where d denotes the distance from the point x to the implicit surface.

In other words, $f(x)$ is a signed distance function if and only if the exact distance from a point x to the surface of the object A is produced from the function. From this, we can begin to implement sphere tracing.

We can take a sphere as an example, which is most commonly used when explaining SDFs. Below is the SDF of a sphere with radius R .

$$f(x, y, z) = \sqrt{x^2 + y^2 + z^2} - R \quad (2.3)$$

Let's do an example with a unit sphere (radius 1), with point a at the origin, $(0, 0, 0)$.

(1)	$f(0, 0, 1) =$	0
(2)	$f(0, 0.25, 0) =$	- 0.25
(3)	$f(1, 1, 1) =$	- 1 + $\sqrt{3}$

For equation 1, the SDF returns a 0 meaning it is on the surface of the sphere. For 2, it is negative, within the sphere. Finally, equation 3 returns a positive value which is that distance away from the surface.

2.2 Finding SDBs with Lipschitz bounded functions

To be able to find SDFs for more complex geometry, we need to introduce Signed Distant Bounds.

A few definitions:

- The function $f(x)$ will be used to define the implicit surface we will calculate the distance to.
- The point a is defined as a point in space where we will calculate the distance to the surface from. Up to this point I have been referring to this point as the virtual observer.
- The point b is defined as a point on the surface of the implicit surface.
- The set B is defined as a set of coordinates along the surface of the implicit surface.
- $d(a, B)$ is defined as the shortest distance from the point a to the implicit surface, and is **the goal of what we are trying to achieve at this stage**.
- It is important to note that there are functions which are *NOT* Lipschitz bounded and therefore this method of finding a signed distance will not work for those functions.

While having the exact distance is optimal, having a shorter distance still works. It just takes more steps to reach the final intersection. The smaller distance generates a smaller radius where there are no intersections, but that radius if it were done optimally would lie inside the radius of the optimal radius where there are no intersections.

We can denote this as:

$$SDB(a) \leq SDF(a) \quad (2.4)$$

The main takeaway from this is that because it is not always possible to find the SDF, the SDB can be used as a suitable alternative.

Now that that's established, we can continue with answering the question above. John C. Hart in his paper [3][pg.4] proposed a method solving this question using a mathematical concept known as the Lipschitz constant, λ .

$$|f(a) - f(b)| \leq \lambda ||a - b|| \quad (2.5)$$

λ is a real valued number that when taking the product of λ with absolute value of $(a - b)$ is greater than or equal to the value of $f(a) - f(b)$. The important bit to note is that it is equal to or greater, meaning when we calculate the Lipschitz constant, we only to find the smallest value of the constant. There is no upper limit to the Lipschitz constant.

We can rearrange this equation, making λ the subject to give:

$$\lambda \geq \frac{|f(a) - f(b)|}{||a - b||} \quad (2.6)$$

To those who have studied single variable calculus, we can see this new equation is similar to the difference quotient, which is written as:

$$\frac{|f(a) - f(b)|}{b - a} \quad (2.7)$$

This represents an average value of f over the interval $[a, b]$, and as a and b become further apart, we get closer to an approximation of the derivative of $f(a)$. But we want the exact, not the derivative. To get the exact, we need to make the difference between a and b very small, writing the equation as:

$$f'(a) = \lim_{||a-b|| \rightarrow 0} \frac{f(b) - f(a)}{b - a} \quad (2.8)$$

We can now combine equations 2.7 and 2.8. λ will be equal to the derivative of the f .

$$\lambda \geq \lim_{b \rightarrow a} \frac{|f(b) - f(a)|}{||b - a||} = |f'(a)| \quad (2.9)$$

But how does this fit into calculating the signed distance function? From Hart's proposal of using the Lipschitz constant, he compares the distance $||b - a||$ to the $d(a, B)$ that we defined earlier. Now looking at the equation:

$$f(a) - f(b), \quad (2.10)$$

if b lies on the surface of the implicit shape then we know $f(b) = 0$. This is the implicit function returning a distance from the surface, and since b is on the surface then the distance is 0. We now know $f(b) = 0$ so we rewrite equation 2.10 as just $f(a)$. [5].

We now take the original proposed equation from Harts, equation 2.3, and we can apply the following:

- $||b - a|| = d(a, B)$
- $f(a) - f(b) = f(a)$

to give us the final solution for the function to be:

$$d(a, B) \geq \frac{f(a)}{\lambda} \quad (2.11)$$

The part to note is the greater than or equal to sign. Whilst the RHS of the equation is equal to $d(a, B)$ it is a signed distance function. However if the RHS is less than the LHS then it is a signed distance bound.

Now that we know that we can find any SDF/SDB, by dividing the implicit surface equation by the Lipschitz constant, we need to find a way to compute λ . Luckily this is pretty straightforward, as we know:

$$\lambda \geq f'(a). \quad (2.12)$$

If λ is at least the maximum value of $f'(a)$ then we just need to compute the function's first and second derivative. We need to find the stationary points of the function and determine whether they are the minima or the maxima. The maxima will be equal to our Lipschitz constant.

2.3 Constructive Solid Geometry

The real power of SDFs and SDBs comes from how we can combine them to construct increasingly complex geometric shapes.

$$f_{A \cap B}(x) = \min[f_A(x), f_B(x)] \quad (2.13)$$

For union, we merge the two simple shapes and the complex shape will be the whole merged shape. To find the intersection with the ray, we just calculate the distance to the closer shape of the two shapes.

$$f_{A \cap B}(x) = M[f_A(x), f_B(x)] \leq d(x, f^{-1}(0)) \quad (2.14)$$

For intersection, things are a little more complicated. This is a signed distant bound, as it provides us not with a distance, but a value that is always upper bounded. In the context of a sphere tracing algorithm, using a SDB works almost just as well, the only cost being an increase in the number of steps to find an intersection.

As a result, we are able to have functions to find the signed distance for very complex shapes such as [3, pg.4] ellipsoids and super-quadrics.

2.4 A simple scene with a plane and a sphere

Let us start with a simple scene consisting of a sphere G which lies on a flat plane K parallel to the x-axis.

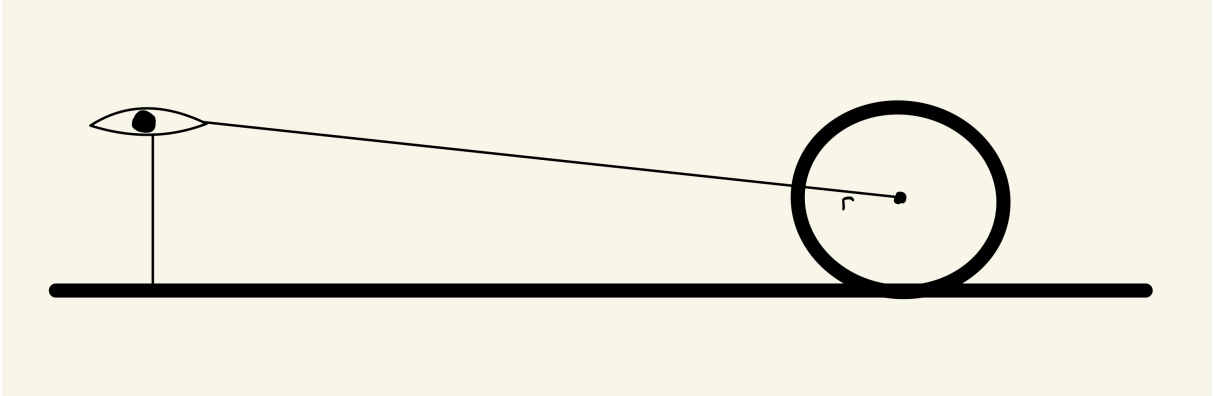


Figure 4: A two-dimensional representation of a ground plane, camera and a sphere.

The distance to the plane is simple, as this is just the height of the camera C subtract the height of the plane.

$$d_1 = (C_y - K_y) \quad (2.15)$$

Then to define the distance to the sphere from the camera, we can first off say that the distance from the centre of the sphere to the camera is:

$$d_2 = G - C, \quad (2.16)$$

where $(G - C)$ denotes the distance between the points via Pythagoras' theorem.

Since this is an overestimate as we are calculating the distance to the centre, we can correct this by subtracting the radius of the sphere from what we have above.

$$d = (G - C) - r \quad (2.17)$$

Now that we have two objects in the scene defined, we need to consider that the ray marching algorithm we defined in the previous section does not have a conception of direction. Therefore we need to take the smallest of the distances from the point to the object to determine the step along the ray march.

$$d = \min(d_1, d_2) \quad (2.18)$$

2.5 Signed Distant Function of a capsule

A capsule can be defined as two spheres connected by line segments from relative circumferences. We can calculate the distance to the capsule by considering another line segment drawn between the centres of the two spheres. Now the distance to the capsule is the distance from the camera point to the closest point on the line segment subtract the radius.

Below is a 2D representation of a capsule with the relevant variables:

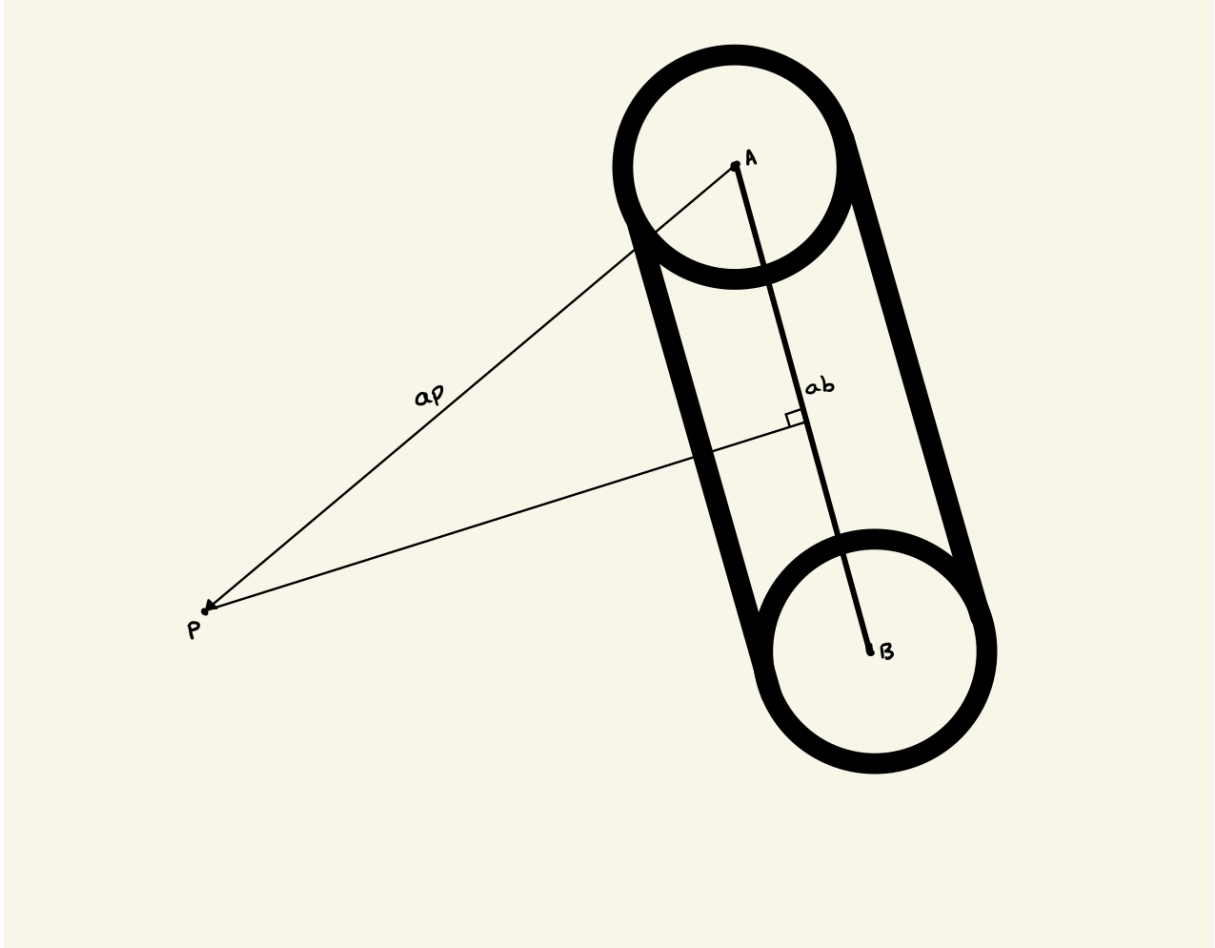


Figure 5: A two-dimensional representation of a capsule

Using vectors, we can get the distance. We call the distance from point p to point A the vector $\vec{ap}$; similarly, the distance from the point A and B the vector $\vec{ab}$. We then take the dot product of the vectors $(\vec{ap}, \vec{ab})$. We need to normalise this vector such that a variable t is more than 1 when below point B and less than 0 when above the point A . To do this, we divide the dot product above by the dot product of $(\vec{ab}, \vec{ab})$.

$$t = \frac{\text{dot}(\vec{ap}, \vec{ab})}{\text{dot}(\vec{ab}, \vec{ab})} \quad (2.19)$$

This will give us what we specified earlier. However, it is not ideal to have t greater than 1 or less than 0. When this is the case, we can use either sphere A or B corresponding with the value of t to determine the distance to the capsule using the same method in determining the distance to the sphere.

Therefore we can clamp the value of t , thus $0 \leq t \leq 1$.

We can then find the point c which is the closest point to p on the line segment.

$$c = A + t \cdot \vec{ab} \quad (2.20)$$

That is all we need to define the SDF of a capsule, which is now:

$$d = (p - c) - r \quad (2.21)$$

2.6 Signed Distant Function of a torus

To imagine how we interpret a torus in terms of signed distant functions, we need to see that a torus is essentially constructed with two circles. The first circle is hollow around the origin and we draw a second circle perpendicular at a point on the radius. We then extend and revolve that circle across the radius of the first circle to generate the torus.

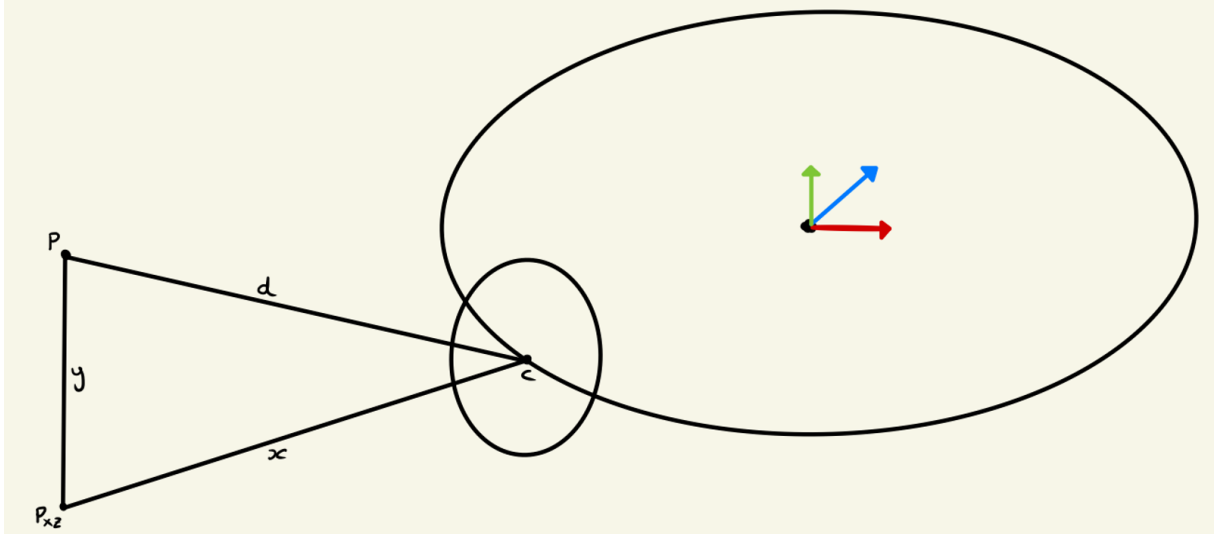


Figure 6: Figure representing a torus SDF

To get the distance from a point to the torus, we need to calculate the distance to the centre of the second circle, then subtract the radius. For this, Pythagorus' theorem is our friend. We can get the distance x , as this is defined as just the distance from the torus subtract the radius of the first circle. The length y is just the Y component of the point P .

$$x = (P_{xz}) - r_1 \quad (2.22)$$

$$y = P_y \quad (2.23)$$

Now that we have x and y , the SDF of the torus can be defined as:

$$d = (\vec{xy}) - r_2 \quad (2.24)$$

2.7 Signed Distant Function of a cuboid

To find the SDF of a cuboid G , we can follow a similar path to how to found the SDF of a torus. We have a cuboid with the centre at the origin, and a point p for which we'll calculate the distance to the cuboid from. Below shows a 2D representation of this.

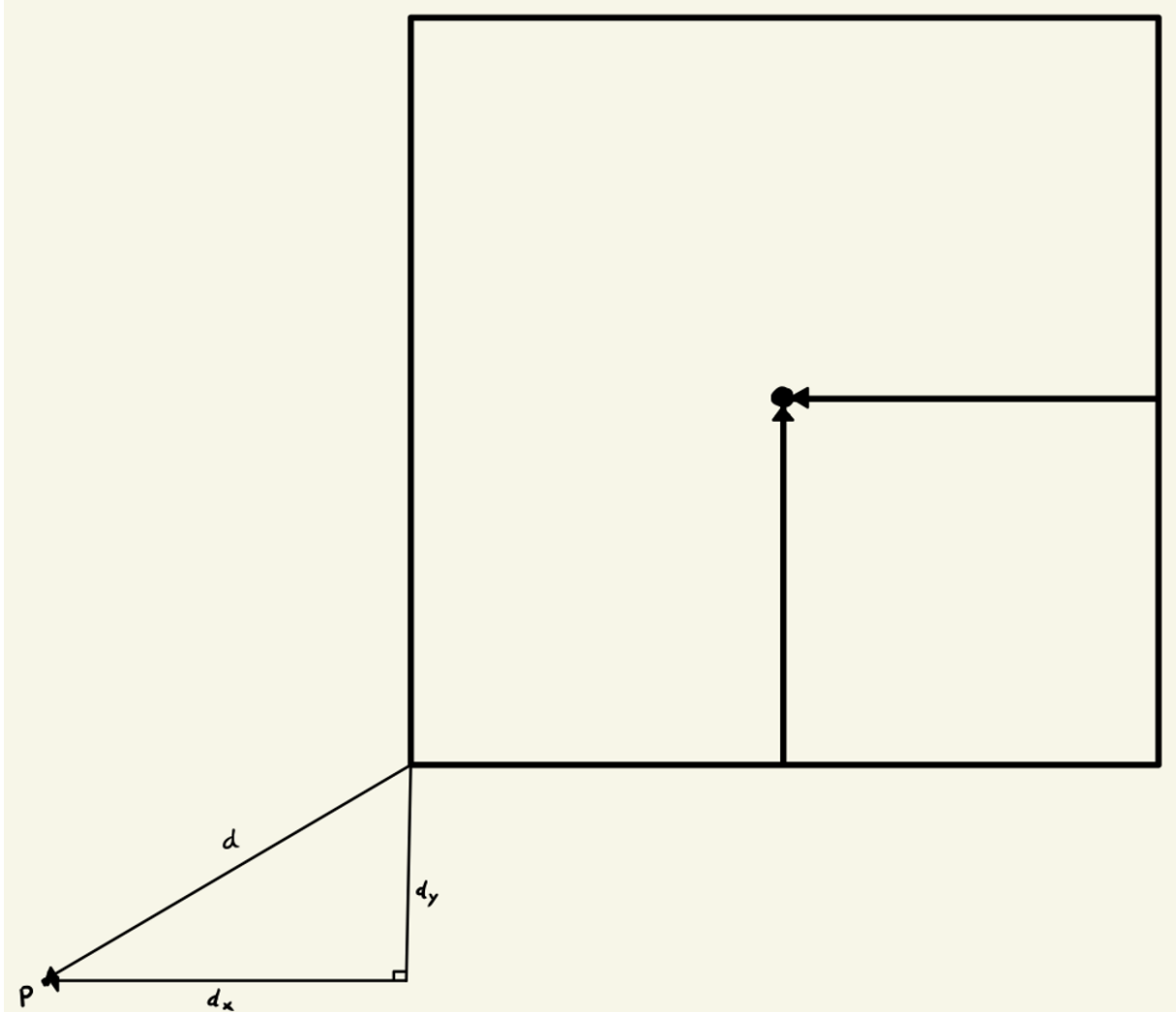


Figure 7: 2D depiction of a cuboid SDF

We once again use Pythagoras' theorem. From the point p , we take the x and y components and subtract half of the respective components of the rectangle.

However, if the closest point to p on the rectangle's surface is not the distance of the hypotenuse, then we need to ignore that hypotenuse and only include the component the leads straight to the square's surface.

$$d_x = \text{abs}(p_x) - \frac{G_x}{2} \quad (2.25)$$

$$d_y = p_y - \frac{G_y}{2} \quad (2.26)$$

Therefore we can define the SDF of a cuboid to be:

$$d = \max(|p| - G, 0) \quad (2.27)$$

where G represents the x, y, z lengths of a cuboid centred at the origin.

2.8 Signed Distant Function of a cylinder

The final SDF I will be introducing will be that of a cylinder. The fortunate part for this is we can use the capsule SDF as a base.

By taking the capsule signed distant function and removing the clamps on the t value, we actually get an infinitely long cylinder, which may be useful for future geometry! However, in this case we require that this object has flat caps at both the top and bottom of the shape so that it forms a cylinder.

To find the distance from a point p , the only thing that differs from the capsule is finding the distance to the non-lateral faces of the cylinder (the lateral area being the curved surface face of the cylinder). We approach finding this area using Pythagoras' theorem again.

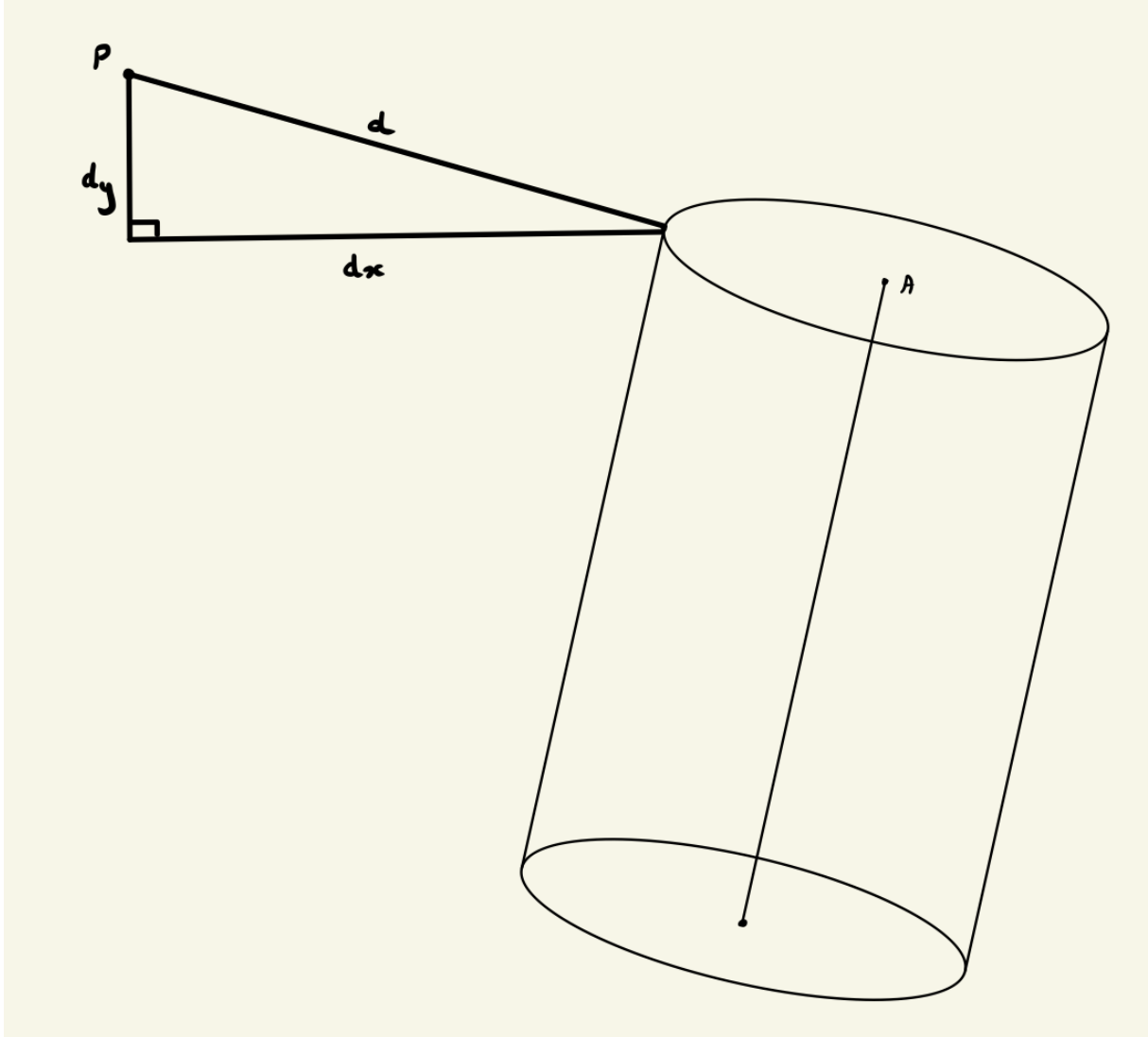


Figure 8: Figure representing a cylinder SDF

The image above shows a three-dimensional cylinder. We draw the triangle as we've done before and we need to find the x and y components of the triangle to be able to find the hypotenuse.

The x component is the same as if we were calculating the SDF for a capsule.

$$x = (p - c) - r \quad (2.28)$$

However, for the y , we need to consider the t value. The t value is clamped between 0 and 1, and if we are able to manipulate this such that the value is 0 at both ends of the cylinder, smaller than 0 inside the cylinder, and larger than 0 on both sides outside of the cylinder, then we can find a normalised y distance from either end of the cylinder. We can solve this by using absolute values.

$$y_{normalised} = |t - 0.5| - 0.5 \quad (2.29)$$

What this essentially does is that it shifts the value of t by 0.5 such that it goes from -0.5 to 0.5 , then we take the absolute value of that so we now consider it to be 0 to 0.5 . We then shift the whole range by 0.5 once again such that it's against 0.

Now this gives us the normalised value of y , but we need the non-normalised form. To do this, we multiply y by the length of $\vec{ab}$.

$$y = |t - 0.5| - 0.5 \times (\vec{ab}) \quad (2.30)$$

Now that we have x and y , we finish up by applying Pythagoras' theorem to calculate the distance from point p to the object.

$$d = \max(\sqrt{x^2 + y^2}, 0) \quad (2.31)$$

2.8.1 Interior distance

We mentioned briefly before that an SDF function can return a negative value if the distance lies within an object. This is an interesting concept as by considering this interior distance, we can get improved estimations of the true distance to the object.

This is by no means a necessary addition to calculating the distance, as when this is applied to generating the objects later on within my ray marching algorithms, we will see little to no improvements at the cost of performance.

But the way to calculate interior distance for a cylinder is as followed:

$$i = \min(\max(d, y), 0) \quad (2.32)$$

Essentially, this takes the largest of the two values (d, y) for any point inside of the object. Since the SDF is negative, this will be the value closest to 0. This is put inside a minimum function, with parameters 0 and the maximum of (d, y) as we only do this for points inside the object and so for any points outside the object, the distance will be capped at 0.

We can apply this technique to the other SDFs, but due to the performance hit I will be limiting myself to be only considering this concept for the cylinder.

3 Illumination and Shading theory

3.1 Introduction

So far we've discussed the idea of sphere tracing and defining geometry in the computer space by calculating the intersections. Once we know what geometry is in the space, we need to illuminate it and give it shading. In this section, we will discuss the shading of the geometry using a handful of interesting techniques. These methods can be combined to create more visually satisfying geometry.

3.2 Lighting

3.2.1 Directional light

Directional light is produced from a source of light which sends rays of light in all directions with equal intensity. Intensity is defined as the rate at which light spreads over a surface area. For more realistic images with varying intensities, we need to consider a different type of light source; namely the point light.

3.2.2 Point light

A point light also shoots rays of light in all directions, but the intensity of the light weakens as the ray travels further from the light source. It is usually a good idea that when creating a scene within computer graphics, we have one main light (i.e. sunlight) as a directional light and smaller lights such as a lamp or bulb as a point light.

3.2.3 Lambertian reflection

Let's consider a point light above a ground plane. This light source can send rays of light directly down onto the plane to illuminate a surface area immediately below. The blue line in figure 9 below the rays represents the illuminated surface. Following this, figure 10 will change the light direction such that it hits the surface at an angle.

We can see now that the blue illuminated surface is much larger. So what we need now is a method to determine the size of the illuminated surface which is dependent on the angle at which the light rays hit the surface.

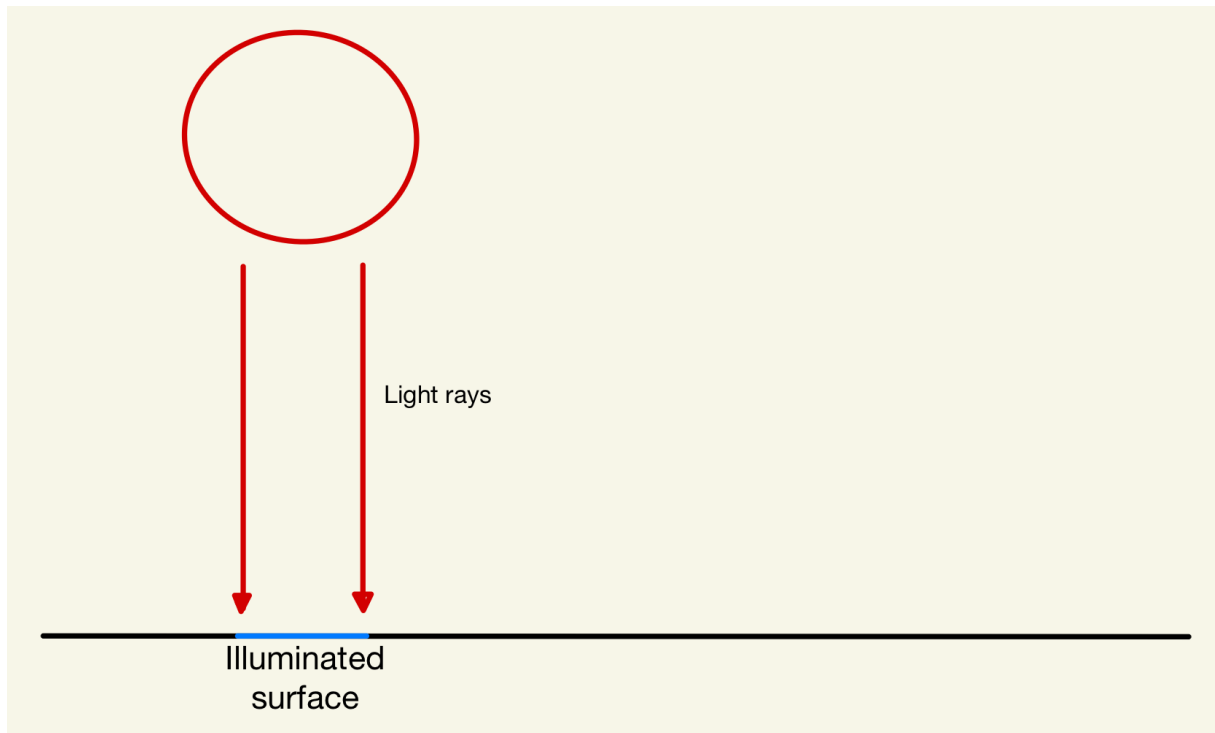


Figure 9: Figure showing a light source with light rays perpendicular to the surface shining towards it.

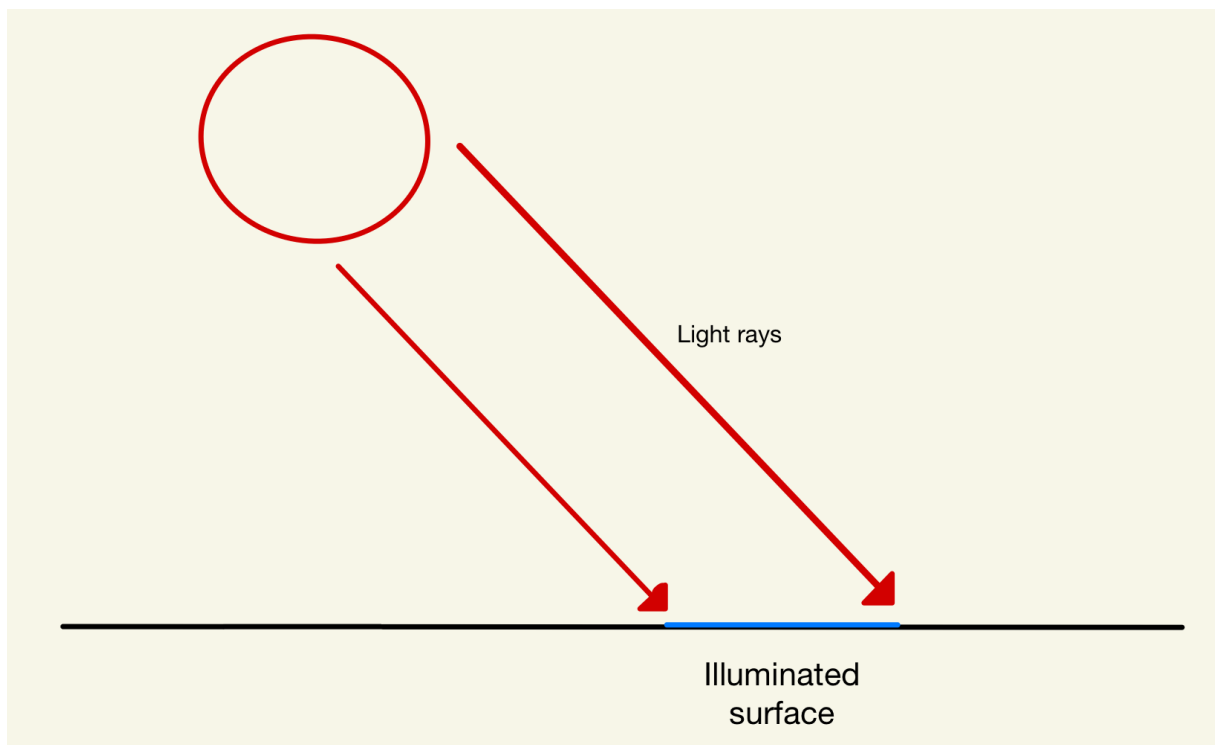


Figure 10: Figure showing a light source with light rays sent towards the surface at an angle.

Fortunately there is a simple way to calculate this by creating two vectors: the light vector which is vector from the light source to the surface, and the normal vector which is the vector which is perpendicular to the surface. Taking the dot product of the two will generate a value which is 1 when the normal vector is equal to the light vector, or will tend to 0 as the light vector becomes more parallel with the surface.

The value of the light vector is easy to determine. It is just the distance between the light source and the surface.

The normal vector however requires more thought. It is possible for the ground plane to set the normal as some constant perpendicular as the ground plane is an $x - axis$ plane, but for more complex geometry it is imperative to have a method to determine the normal to the surface.

What we can do is compute the normal to the surface by computing the gradient of the SDF at particular points on the surface. This gradient is orthogonal to the level curve at that point. [11]

$$n = \nabla f(p) \quad (3.1)$$

A classical method of calculating the gradient is by using a finite difference method. One of these is the central difference method. We can define the gradient as:

$$\nabla f(p) = \left(\frac{\partial f(p)}{\partial x}, \frac{\partial f(p)}{\partial y}, \frac{\partial f(p)}{\partial z} \right). \quad (3.2)$$

We can define each of these partial differentials by the central difference:

$$\frac{\partial f(p)}{\partial x} \approx \frac{f(p + h, 0, 0) - f(p - h, 0, 0)}{2h}, \quad (3.3)$$

where h is the difference between two sample points on the surface.

The importance of a central difference instead of a forward or backwards is that we have no directional bias when computing the normals.

Considering each of these partial derivatives, the normal is calculated by:

$$n = \begin{pmatrix} f(p + h, 0, 0) - f(p - h, 0, 0), \\ f(p + 0, h, 0) - f(p - 0, h, 0), \\ f(p + 0, 0, h) - f(p - 0, 0, h) \end{pmatrix}$$

Now that we have n , it is required to normalise this normal vector, along with normalising the light vector.

We finally take the dot product of the light and normal vectors to get the lighting model.

This is all called the Lambertian reflection, which is a type of diffuse reflection. This light when hitting the surface is no longer distributed uniformly on that surface, giving a more realistic image, as the illumination will vary around the point where the light intersects.

3.3 Shadows

Shadows within ray marching vary in complexity depending on what types of shadows we are trying to achieve.

The basic idea of shadows is:

$$S = \begin{cases} 0, & \text{if the light is blocked;} \\ 1, & \text{if the light is not blocked.} \end{cases} \quad (3.4)$$

The method I'll be implementing to determine whether a point is blocked from light or not is by doing a ray march from that point to the light source. We know the distance to the light source, as this is just the vector between the light position and the point in the shade. So if the ray march to the light source returns a value smaller than the distance to the light source, then we know we've struck an object in the space. We can then give a shadow to that point.

This is a very basic model for shadows, and there are more technical methods which provide better results (i.e. more realistic shadows) but this is usually at the cost of performance.

3.4 Materials

All the geometry defined in the space can also be defined to have its own material. This means that they can have their own colour and reflectivity.

To apply a material to each of the SDFs, we need a way to determine which piece of geometry the ray of light is intersecting with. Well this is the same way we calculate the distance to the nearest object the light ray is intersecting with, plus an extra step.

By comparing the closest distance of all distances to every object in the scene, we can determine which object the ray intersected with. This is more apparent when applying it to programming, as the application of the materials relies on conditional statements. Therefore, this will be expanded upon in the next section under the implementation of materials.

3.5 Reflections

Reflections within computer graphics can be a relatively computationally heavy task. This is because multiple reflective geometry in a scene can have an exponentially growing number of reflections.

3.5.1 Recursive reflections

The way I plan to implement reflections within this project is via recursive ray marching. After finding an intersection, we can use the normal to find the reflected ray direction and do a ray march. This reflected vector will find a new intersection to map a pixel as the texture for the original intersection.

Recursion is the idea of repeating a certain procedure, and in programming this usually involves a method or function calling itself. The beauty of this is that we can minimise repeated code and reduce the computational stress of running excessive lines of code, which is great considering the aforementioned exponential growth in the complexity of computing multiple reflections.

3.5.2 Specular reflection (Phong illumination model)

On any surface with some level of shine, we can observe an effect known as *specular reflection*. In real life, take a shiny, optionally non-reflective (a non reflective surface as an example shows this effect the best), and shine a white light onto it. The central point where the light intersects the surface will have a white highlight. This highlight's colour is correspondent with the colour of the source light.

Phong Bui-Tuong [2][pg.729] created an illumination model to implement this specular highlight. For his model, we need two vectors, a reflection vector and a viewpoint vector, both normalised.

To calculate the reflection vector, $\vec{r}_r$, we need to mirror the light vector about the normal of the surface. This is shown in figure 11.

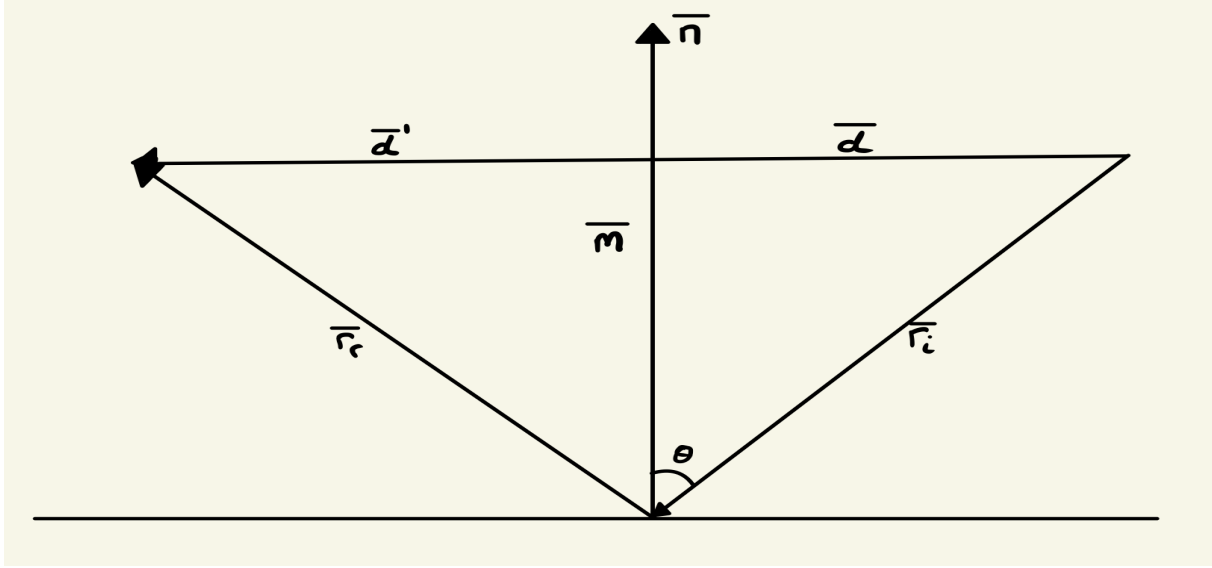


Figure 11: Figure showing a light vector being reflected at the normal, with labels.

We need to find the vector $\vec{r}_r$, which is equal to $\vec{m} + \vec{d}'$.

An expression for $\cos \theta$ is:

$$\cos \theta = \frac{|\vec{m}|}{|\vec{r}_i|} \quad (3.5)$$

$$\therefore |\vec{m}| = |\vec{r}_i| \cos \theta. \quad (3.6)$$

So now we need an expression for $\vec{r}_i$. The scalar dot product of $\vec{r}_i \cdot \vec{n}$ is:

$$\vec{r}_i \cdot \vec{n} = |\vec{r}_i| |\vec{n}| \cos (\pi - \theta) \quad (3.7)$$

We can expand and simplify $\cos (\pi - \theta)$.

$$\cos (\pi - \theta) = \cos \pi \cos \theta + \sin \pi \sin \theta \quad (3.8)$$

$$\cos (\pi - \theta) = -1 \cdot \cos \theta + 0 \cdot \sin \theta \quad (3.9)$$

$$\cos (\pi - \theta) = -\cos \theta \quad (3.10)$$

$$\vec{r}_i \cdot \vec{n} = -|\vec{r}_i| |\vec{n}| \cos \theta \quad (3.11)$$

Now assuming $|\vec{n}| = 1$,

$$|\vec{m}| = -\vec{r}_i \cdot \vec{n} \quad (3.12)$$

$$\vec{m} = |\vec{m}| \cdot \vec{n} \quad (3.13)$$

Using the fact that $\vec{d}'$ is congruent to $\vec{d}$, and that

$$\vec{d} = \vec{r}_i + \vec{m}, \quad (3.14)$$

we can combine all this to find $\vec{r}_r$.

$$\vec{r}_r = \vec{m} + \vec{d} \quad (3.15)$$

$$= \vec{m} + \vec{r}_i + \vec{m} \quad (3.16)$$

$$= \vec{r}_i + 2\vec{m} \quad (3.17)$$

$$= \vec{r}_i - 2(\vec{r}_i \cdot \vec{n})\vec{n} \quad (3.18)$$

With this reflection vector, we can take the scalar dot product of it with the negative of the viewpoint which is just the negative ray direction. The final Phong contribution is:

$$\alpha = \text{dot}(\vec{r}_r, -\vec{r}d) \quad (3.19)$$

3.5.3 Fresnel term

The Fresnel term is a response originated by Augustin-Jean Fresnel which applies some physical basis to the ray marching algorithms. What this intends to do is change the lighting slightly when looking at the object at different angles, particularly close to parallel to a tangent to the point on the surface.

Shlick's approximation for the Fresnel term is written as:

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos(\theta))^5, \quad (3.20)$$

$$\text{where } R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2. \quad (3.21)$$

R_0 is the reflection coefficient for incoming light that is parallel to the normal, which is used to calculate R which is the Fresnel term. This R_0 actually becomes 0 when we simplify the air refractive index to be 1 and consider the surface refractive index of each of the objects we have SDFs for to be 1 also. We can therefore simplify the Fresnel contribution to be:

$$R = (1 - \cos \theta)^5. \quad (3.22)$$

When implementing this term into our ray marching, we note that:

$$\cos \theta = \text{dot}(n, -r), \quad (3.23)$$

where n is the normal vector and r is the ray direction.

4 Implementation with GLSL

4.1 Introduction to GLSL and Shadertoy

Sphere tracing has become a popular technique for rendering images directly on a graphics processing unit (GPU).

All the methods and techniques I have discussed so far can be implemented by a programming language known as OpenGL Shader Language (GLSL) [9]. The syntax is very similar to C as it is based off it, so therefore it is considered a high-level language. This means that the syntax used in the language more closely resembles common English rather than low-level language which more machine oriented. Shaders are programs that the GPU executes to render three-dimensional scenes. A shader written in GLSL is what I hope to reflect as the goal of this project.

There are many implementations of GLSL widely available. However I intend to utilise a specific implementation which can be run within a web browser. The name of the implementation is ShaderToy, and it both an integrated development environment (IDE) and a community for many who enjoy sharing open-source shaders. Shadertoy can be accessed via the web (www.shadertoy.com) and hosts a variety of community written shaders. These shaders vary in techniques, and there are many other methods of generating computer graphics available besides sphere tracing, or ray marching.

We can begin a project by creating an account and clicking *New* at the top right. This loads a default shader that we can begin to edit.

4.2 Ray Origin and Ray Direction

We need to first define the ray origin. This is also known as the camera or virtual observer. Within GLSL, we can write this as:

```
vec3 ro = vec3(0, 3, -3);
```

The (x, y, z) coordinates represent the coordinates at which the ray origin is located. I have raised the y by 3 and moved back the z by 3 to increase the field of view (FOV), which allows the camera to show more geometry in the space at a time.

We then need to define the ray direction. At this point, we also normalise the coordinate space such that the origin is in the centre of the screen, as by default the origin is offset.

```
vec2 uv = (fragCoord-.5*iResolution.xy)/iResolution.y;  
  
// Normalising UV coordinates
```

Once the UV is normalised, we can call the function.

```
vec3 rd = defrd(uv, ro, vec3(0, 2, 6), .5);  
  
//Call defrd function to define ray direction  
  
vec3 defrd(vec2 uv, vec3 p, vec3 l, float z)  
  
// l is the rotation point in space
```

```

{

    vec3 f = normalize(l-p), //normalise distance from l to p

    r = normalize(cross(vec3(0,1,0), f)), //calculate cross product of
        vectors f and (0,1,0)

    u = cross(f,r), //cross product again

    c = f*z, //zoom in z direction

    i = normalize(c + uv.x*r + uv.y*u); //formulating ray direction

    return i;

}

```

This function takes in a vector look-at position and a zoom and returns the normalised ray direction with those parameters.

The ray and camera is now defined. The next steps are to create the sphere tracing function and then define the geometry.

4.3 The sphere tracing function

In section 1, I covered a sphere tracing algorithm from Hart's paper. The following is a modified version of the pseudocode written in GLSL.

It takes as inputs the ray origin and ray direction, and outputs a vector containing the distanced travelled and the material of the object intersected. As previously explained, the sphere tracing function marches along the ray in the ray direction from the origin in increments of the shortest distance to an object. Once the distance to travel along the ray becomes very small, smaller than a pre-determined value, then we assume that we have an intersection. Otherwise, we continue to step along the ray until we reach a maximum number of steps which indicates that there is no intersection and we stop the ray march.

```

vec2 SphereTrace(vec3 ro, vec3 rd)

//Sphere tracing function

{

    float d0 = 0.;

    //define distance travelled from origin

    vec2 dSMat = vec2(0);

    for(int i=0; i<MAX_STEPS; i++)

        //for an integer i, while i is less than the maximum steps (defined),
            and increment i

```

```

{

    vec3 p = ro + rd*d0;

    //calculate point p, which is the distance travelled in that single
        marching step

    dSMat = GetDist(p);

    //calculate distance to scene with GetDist function

    d0 += dSMat.x;

    //redefine distance from the origin by adding the distance to the scene

    if(d0>MAX_DIST || dSMat.x<SURF_DIST) break;

    //leave loop if we reach some maximum distance (no hit) or if the
        distance is so small we can classify as a hit

}

return vec2(d0, dSMat.y);

}

```

In the loop, we see the function *GetDist* called. This function can be viewed in the appendix, and essentially this calls all of the defined SDFs to get the distance to every object in the scene from the current point on the ray.

The function then returns a vector with the total distance travelled to the intersection along with the material of the intersected object (if any).

4.4 Implementation of SDFs

Once the sphere tracing algorithm is in place, we can begin to define each of the geometry within Shadertoy.

Firstly, we need to determine how the shader calculates the minimum distance to an object. We do this by determining the distance from the current point to all the defined geometry in the space, using the sphere tracing function.

To do so, we define the SDFs in their own functions [7], and then call them inside a new function so we can compare the distance returned by the SDF function to the other geometry.

The following sections will show how each of the SDFs are written within GLSL.

4.4.1 Sphere on ground plane

```
vec4 s = vec4(2, 1, 6, 1); //define sphere, position xyz and radius r

float sphereDist = length(p-s.xyz)-s.w; //calculating distance to the sphere

float plane = p.y; //define ground plane

d = min(plane, sphereDist); //minimum distance to ground plane or sphere
```

The sphere is defined at point (2, 1, 6) and is given a radius of 1. The variable *sphereDist*, the distance from the point on the ray to the closest point that lies on the surface of the sphere, is then calculated by the *length* operator. This returns the distance from the point *p* to the centre of the sphere, which is an over-calculation as we need the closest point on the surface. To correct this, we subtract the radius from the length to get the distance to the sphere's surface. To show the sphere in the space, we calculate the minimum distance to either the ground plane or the sphere.

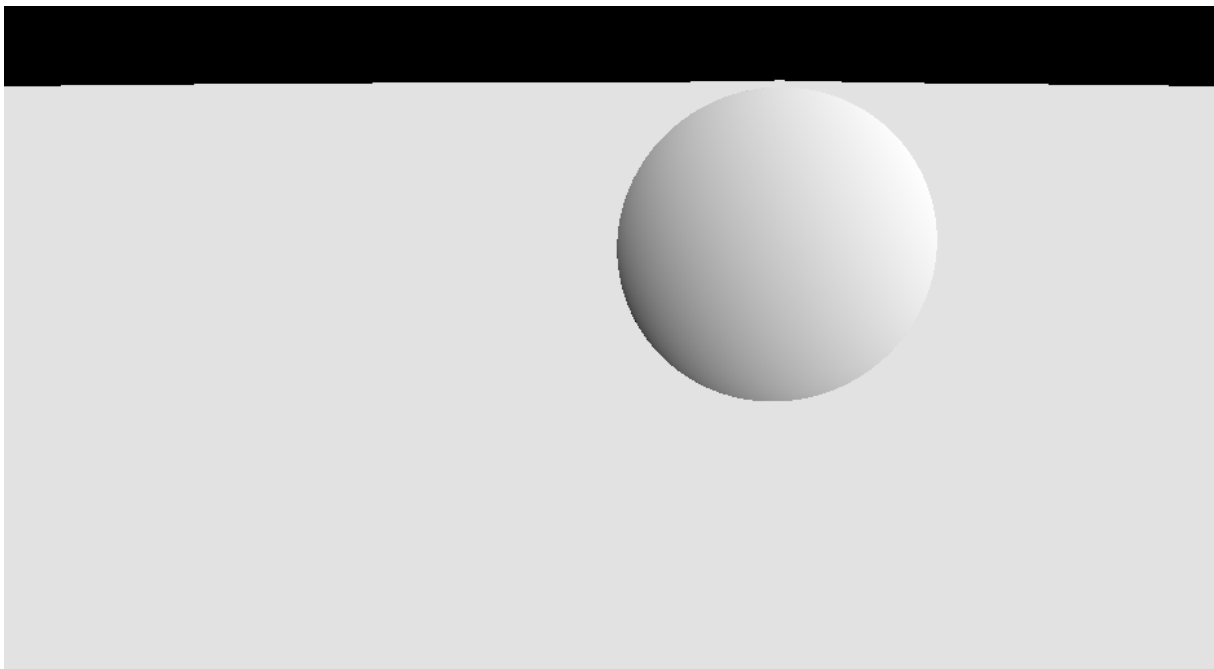


Figure 12: Figure showing a sphere on a ground plane

4.4.2 Capsule

```
float capDist = sdCapsule(p, vec3(0, 1, 6), vec3(0, 2, 6), 0.5);

//calculating distance to the capsule

float sdCapsule (vec3 p, vec3 a, vec3 b, float r)
{
    vec3 ab = b-a;

    vec3 ap = p-a;

    float t = dot(ab, ap) / dot(ab, ab);

    t = clamp(t, 0., 1.);

    vec3 c = a + t*ab;

    return length(p-c) - r;
}
```

The distance from the ray to the surface of the capsule is returned by the function *sdCapsule* with the parameters p , the point on the ray, a , the midpoint on the first circle at the top of the capsule, b , the midpoint on the second circle at the bottom of the capsule, and r , the radius of both circles. As aforementioned, to calculate the closest point on the surface of the capsule, we need to define the vectors $\vec{ab}$ and $\vec{ap}$, so that we can calculate t which is relative to the point on the line AB such that it is above 1 when the point is above the capsule, below 0 when the point is below the capsule, and between 1 and 0 when the point is adjacent to the capsule. We can then clamp t between 1 and 0, before calculating the point c which is the closest point to p on the line segment AB . Subtracting the radius r from this returns the distance to the capsule.



Figure 13: Figure showing a capsule

4.4.3 Torus

```
float torDist = sdTorus(p-vec3(-2,.5, 6), vec2(1.,.5));

//calculating distance to the torus

float sdTorus (vec3 p, vec2 r)
{
    float x = length(p.xz)-r.x;

    return length(vec2(x,p.y))-r.y;
}
```

To implement a torus SDF in GLSL, we define a variable x which is the distance to the torus centre subtract the radius. With this, we can return the distance to the centre of the extended circle on the radius of the torus subtract the radius of the second circle to get the distance to the torus.

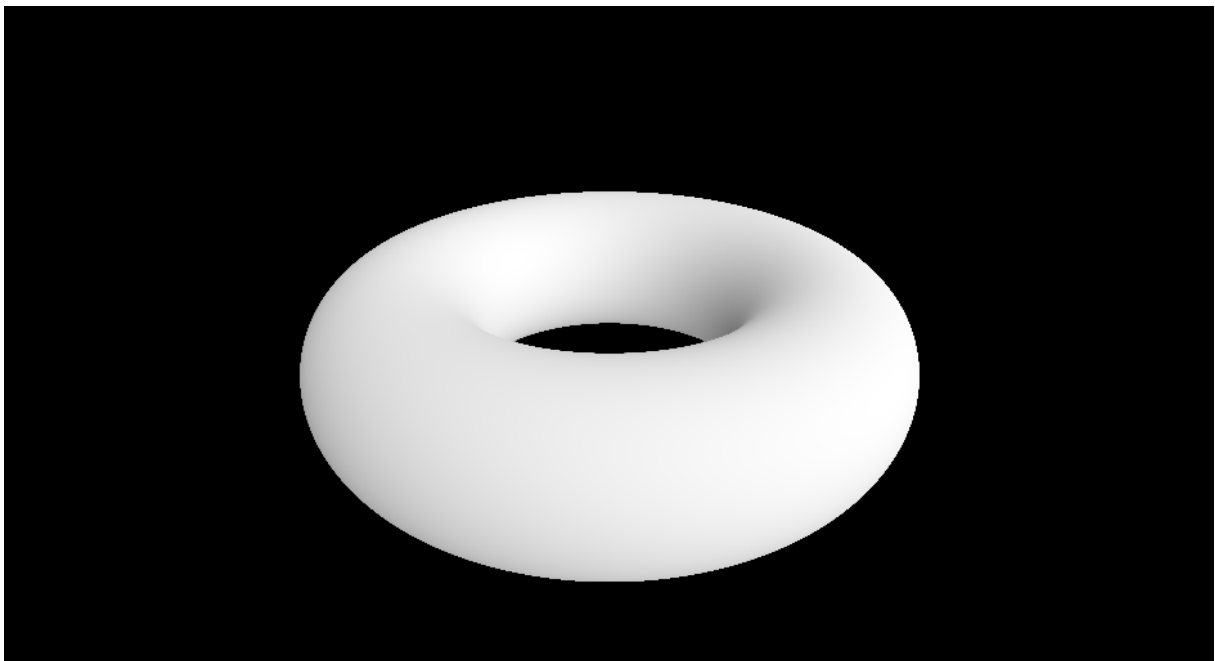


Figure 14: Figure showing a torus

4.4.4 Cuboid

```
float boxDist = sdBox(p-vec3(-2,1,12), vec3(2,4,2));

//calculating distance to the box

float sdBox(vec3 p, vec3 b)
{
    return length(max(abs(p)-b,0.));
}
```

To implement a cuboid in GLSL, we can simply return the maximum distance between the point p and the coordinates of the cuboid relative to the origin. We return the maximum of this and 0 to allow for points where the closest point on the ray has any axis component equal to 0.

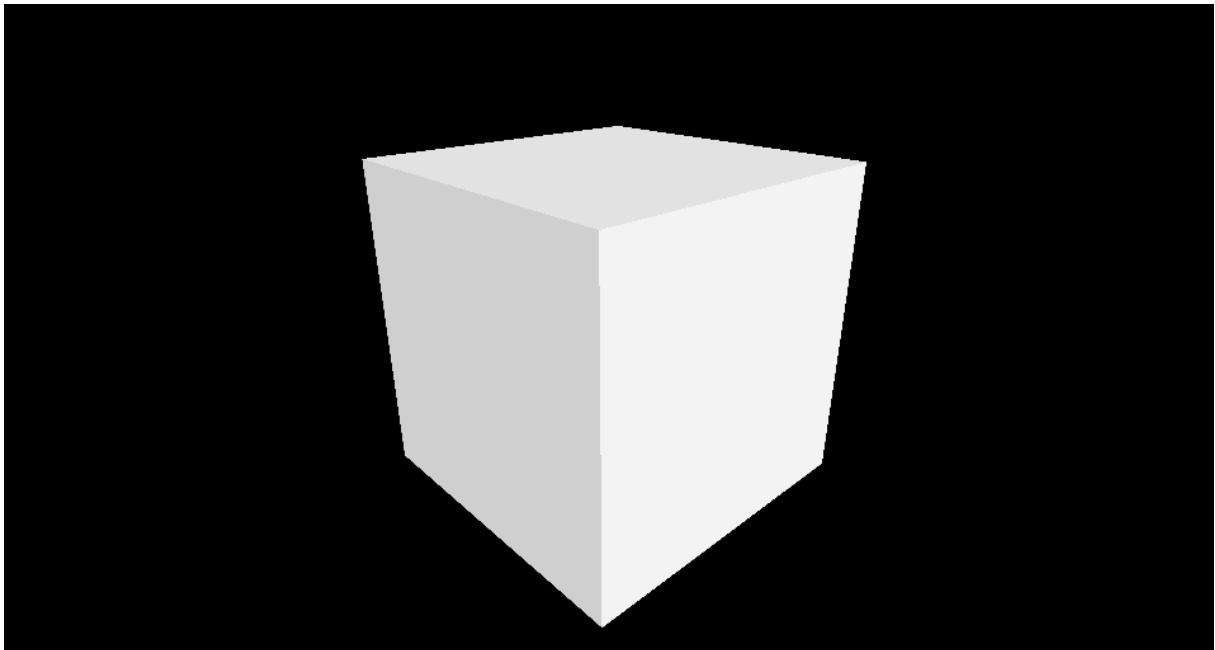


Figure 15: Figure showing a cuboid

4.4.5 Cylinder

```
float cylDist = sdCylinder(p, vec3(0, .3, 3), vec3(3, .3, 5), .3);

//calculating distance to the cylinder

float sdCylinder (vec3 p, vec3 a, vec3 b, float r)
{
    vec3 ab = b-a;

    vec3 ap = p-a;

    float t = dot(ab, ap) / dot(ab, ab);

    //t = clamp(t,0.,1.);

    vec3 c = a + t*ab;

    float x = length(p-c) - r;

    float y = (abs(t-.5)-.5)*length(ab);

    float e = length(max(vec2(x,y),0.));

    float i = min(max(x,y), 0.);

    //gives performance penalty, interior distance to improve step
    estimations

    return e+i;
}
```

This largely follows suit to how we implemented the SDF of a capsule, but for the differences it is fortunate we can apply the differences mentioned in section 2.8 word for word. We also consider interior distance i here, and we can return the overall distance by adding together this interior distance to the distance e calculated by the function.

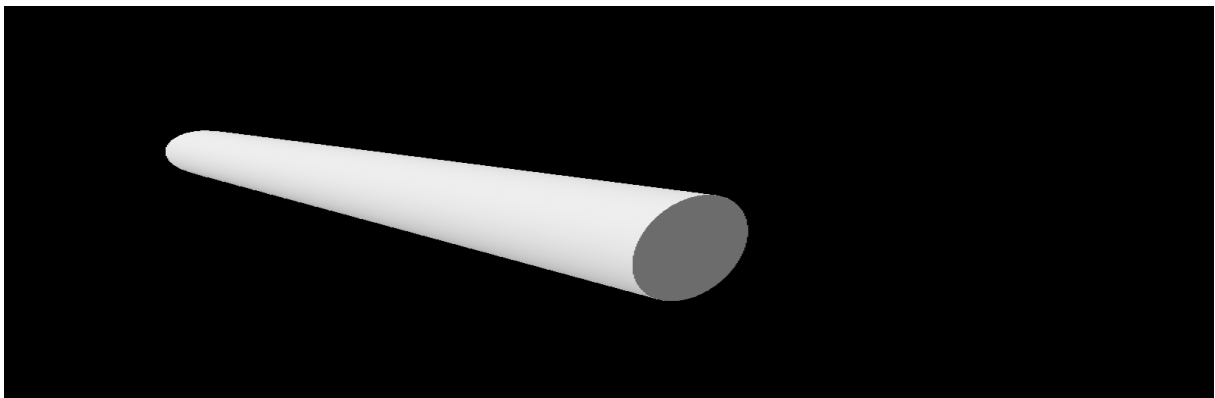


Figure 16: Figure showing a cylinder

4.4.6 Showing the geometry within shadertoy

To be able to show all the geometry within shadertoy, we need a function to get the distances to all the SDFs we have just defined. Below is GLSL code to load in the minimum distances to each of the SDFs.

```
float d;  
  
d = min(capDist, planeDist);  
  
d = min(d, torDist);  
  
d = min(d, sphereDist);  
  
d = min(d, boxDist);  
  
d = min(d, cylDist);
```

4.5 Implementation of shadows

To implement the simple shadow model with diffused lighting, we need to create a function to add a point light to light the scene. Below is a function which returns defines the position of the light source, and rotates it using the run time of the shader as an input for the sin and cos functions.

The diffused light is then calculated by taking the dot product of the normal vector and the normalised light vector, clamped between 0 and 1 which is a preventative measure to stop issues with complex materials. We then ray march from a point just off the surface so that the ray march doesn't end immediately. We end the function by drawing the shadow if the distance returned by the sphere tracing function is less than the distance between the light and the surface, otherwise we return the diffused light. By default, it is very bright so we half it to give the scene a darker look.

```
float GetLight (vec3 p)
{
    vec3 lightPos = vec3(5, 20, -5);

    //position of light source

    lightPos.xz += vec2(sin(iTime), cos(iTime))*10.;

    //moving the light source around the object

    vec3 l = normalize(lightPos-p);

    vec3 n = GetNormal(p);

    //creating the light and normal vectors to calculate the shading dot
    product

    float dif = clamp(dot(n, l), 0., 1.);

    //Calculate the dot product to know the level of shading.

    //Clamped between 0 and 1 as -1 is a potential result which causes
    issues on complex materials

    vec2 d = SphereTrace(p+n*SURF_DIST*2., 1);

    //Ray March from the bottom of the shadow to the light source

    //but moving the point away from the surface a little bit prevents the
    ray march from ending right away

    if(d.x<length(lightPos-p)) dif *= .5;

    //if the distance is less than the distance between the light and the
    surface position

    //then we know that we are drawing the shadow

    //the brightness of the shadow denoted by the dif *=.5

    return .5*dif;
}
```

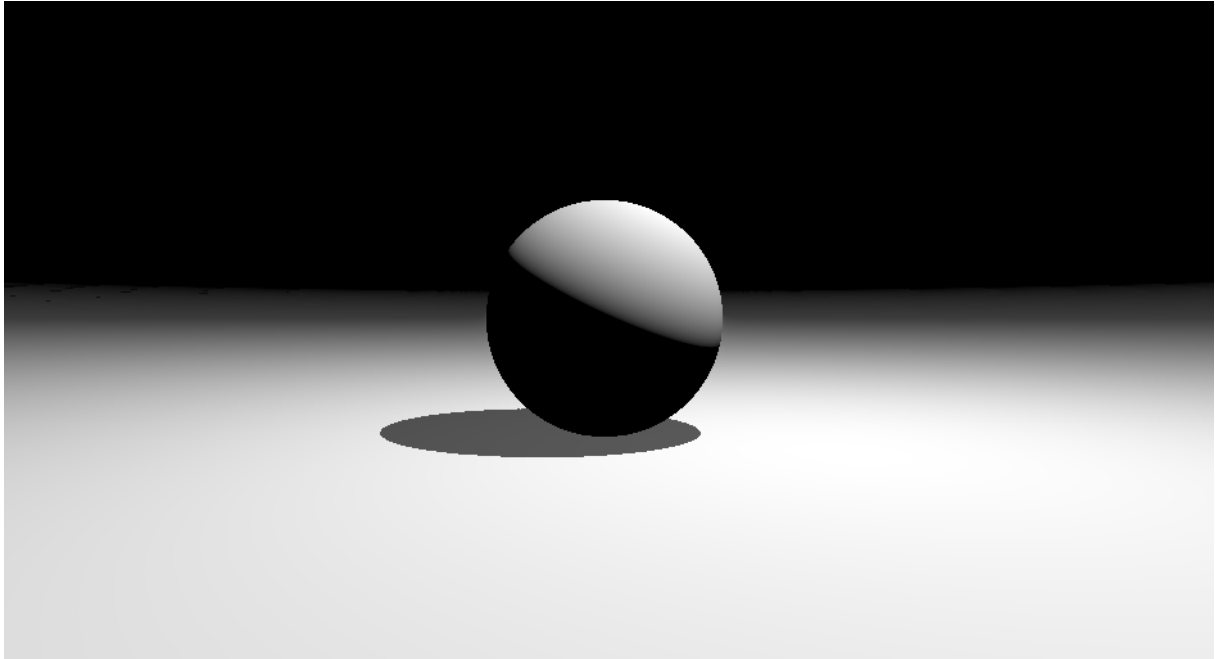


Figure 17: Figure showing a shaded sphere

4.6 Implementation of materials

We can get the shader to recognise which object in space the ray has intersected with to determine the colour of the shape.

```
const int MAT_cuboid=1;

const int MAT_torus=2;

const int MAT_sphere=3;

const int MAT_capsule=4;

const int MAT_cylinder=5;

const int MAT_plane=6;


if (d==boxDist)

mat = MAT_cuboid;

else if (d==torDist)

mat = MAT_torus;

else if (d==sphereDist)

mat = MAT_sphere;
```

```

else if (d==capDist)

mat = MAT_capsule;

else if (d==cylDist)

mat = MAT_cylinder;

else if (d==planeDist)

mat = MAT_plane;

```

This assigns a value to a variable *mat* depending on which object was intersected. At the top is an index which converts integers to strings which are easier to read and refer to.

We can return this *mat* variable out of the function, and use it in a select structure to determine the RGB value of the intersected object.

```

if(mat==MAT_cuboid)

col *= vec3(0.5,1,1);

else if (mat==MAT_sphere)

//col *= ref; //vec3(0,0,1);

else if (mat==MAT_torus)

col *= vec3(1,0,1);

else if (mat==MAT_capsule)

col *= vec3(.5,0,0);

else if (mat==MAT_cylinder)

col *= vec3(0,.5,1);

else if (mat==MAT_plane)

col *= vec3(0,0,0);

```

Shadertoy also allows us to add in a cubemap as a background. There are a few maps readily available, and I've loaded one into *iChannel0* at the bottom right by selecting it, and running the following line.

```

vec3 col = texture(iChannel0, rd).rgb;

```

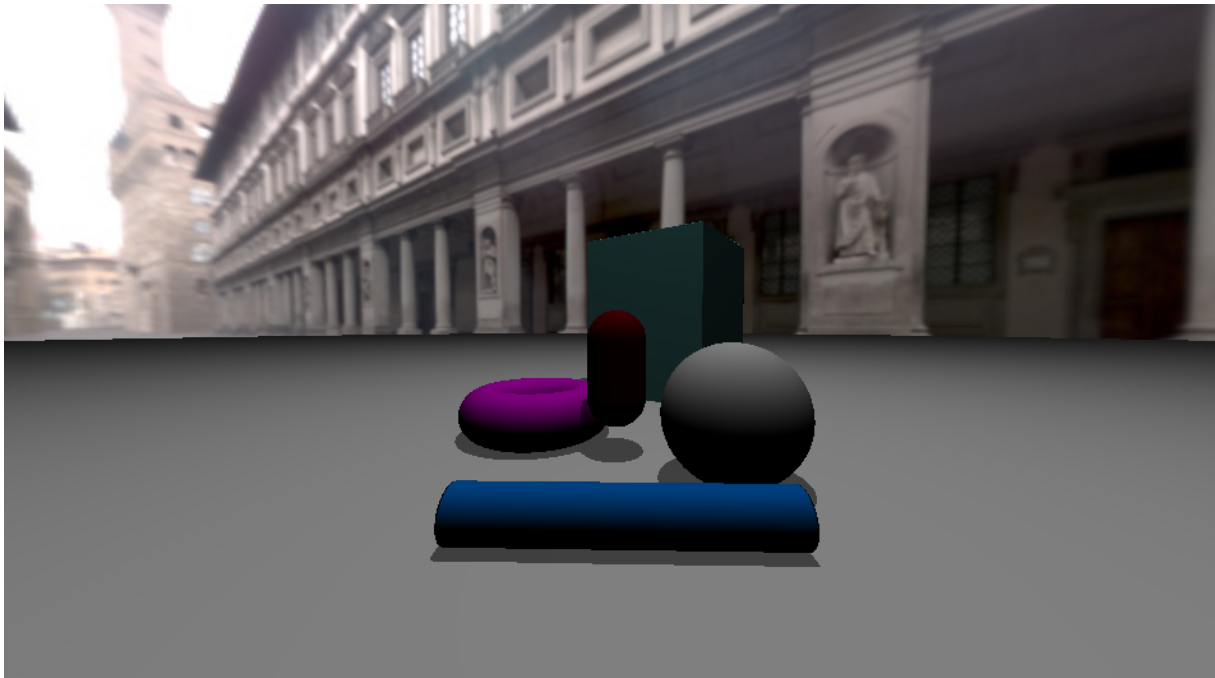


Figure 18: Figure showing all the geometry with colours and a background

4.7 Implementation of Phong's illumination

To add Phong's specular term to the shader, we need to just modify what is returned by our `getLight` function we defined above.

We must first define two vectors; a viewer vector and a reflected light vector. We can call the viewer vector in the function, as it is the ray direction, and we can define the reflected light vector within the function. Below is a snippet we can add to the function above to consider these two vectors.

```
float GetLight (vec3 p, vec3 viewer)
{
    vec3 refl = 2.*dot(n, l)*n - l;
}
```

Now that the two vectors are defined, we modify what we return from the function. Phong's contribution can be added into GLSL by taking the dot product of the two previously defined vectors to a high power and then clamping the value between 0 and 1. The summation of this term with the diffused light gives us the specular effect we are after.

```
return .5*dif + 0.75*pow(clamp(dot(refl, -viewer), 0., 1.), 30.0);
```

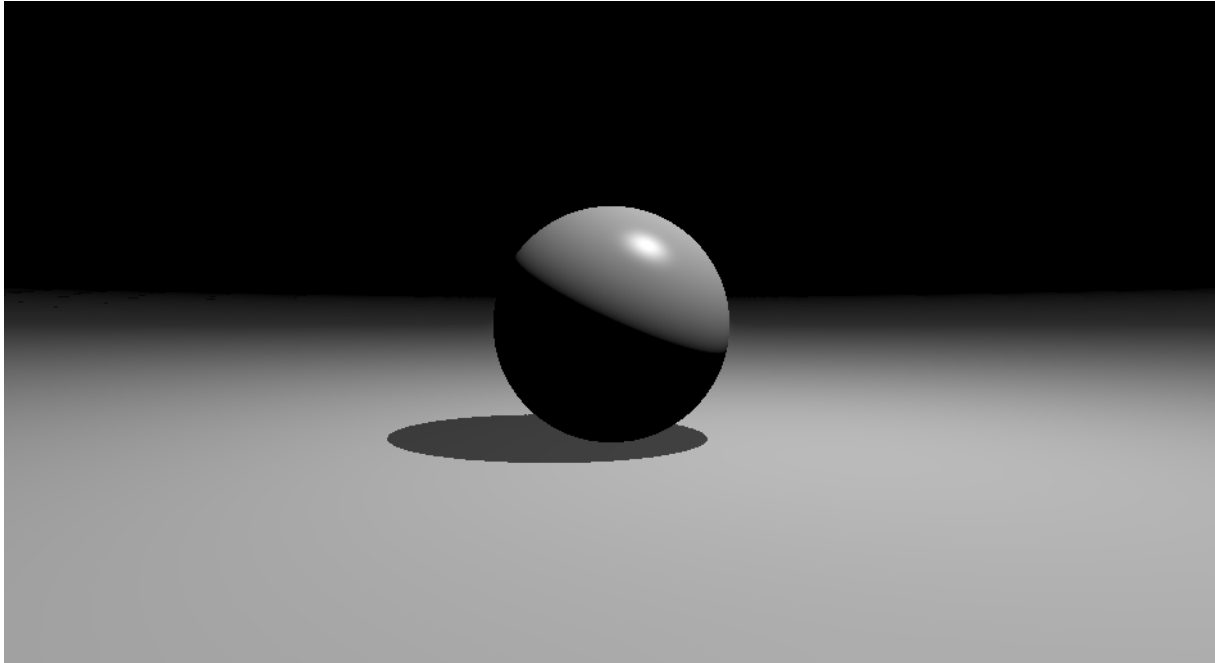


Figure 19: Figure showing a shaded sphere with a specular reflection

4.8 Implementation of reflection

The final piece of building this shader is adding in true reflections. In most programming languages, it is possible to do this with recursion. The idea behind this is that at the intersection, we need to ray march along a reflected vector relative to the normal of the intersection, and essentially rerun the entire shader to generate a reflection on the surface of the object. With recursion, this becomes straight forward as all that we need to do is define a function that can call itself to calculate both the original ray intersection and the reflected intersections. The problem however is that GLSL does not support recursions. We must tackle this differently.

```
#define NUM_OF_REFLECTION 2

vec3 refFact = vec3(0);

vec3 filt = vec3(1);

vec3 col = Render(ro, rd, refFact);

for(int i=0; i<NUM_OF_REFLECTION; i++)
{
    filt *= refFact;

    vec3 reflected = filt*Render(ro, rd, refFact);

    col += reflected*.75;
}
```

The snippet above runs an *for* loop for the number of predetermined bounces and runs the *Render* function (see appendix for the function) with the new ray origin and ray direction. This new ray is calculated within the *Render* function and is changed in the entire program. [8]

With this, we have true reflections for every object in scene. But in reality, not every object has the same level of reflectivity. The *refFact* variable helps simulate this.

When defining the materials of each object, we can now add the *refFact* variable to material. The snippet below shows this for the cuboid.

```
if(mat==MAT_cuboid)

{

    col *= vec3(0.5,1,1);

    refFact = vec3(.1);

}
```

This gives the cuboid a reflectivity factor of 0.1. In real life, there is no object which reflects 100% of light, and no object which reflects no light either. Therefore, it is a good idea in GLSL not to give objects a reflective factor of 1, but due to how GLSL is constructed, it can be okay to give objects 0 reflectivity due to aliasing issues with some shaders.

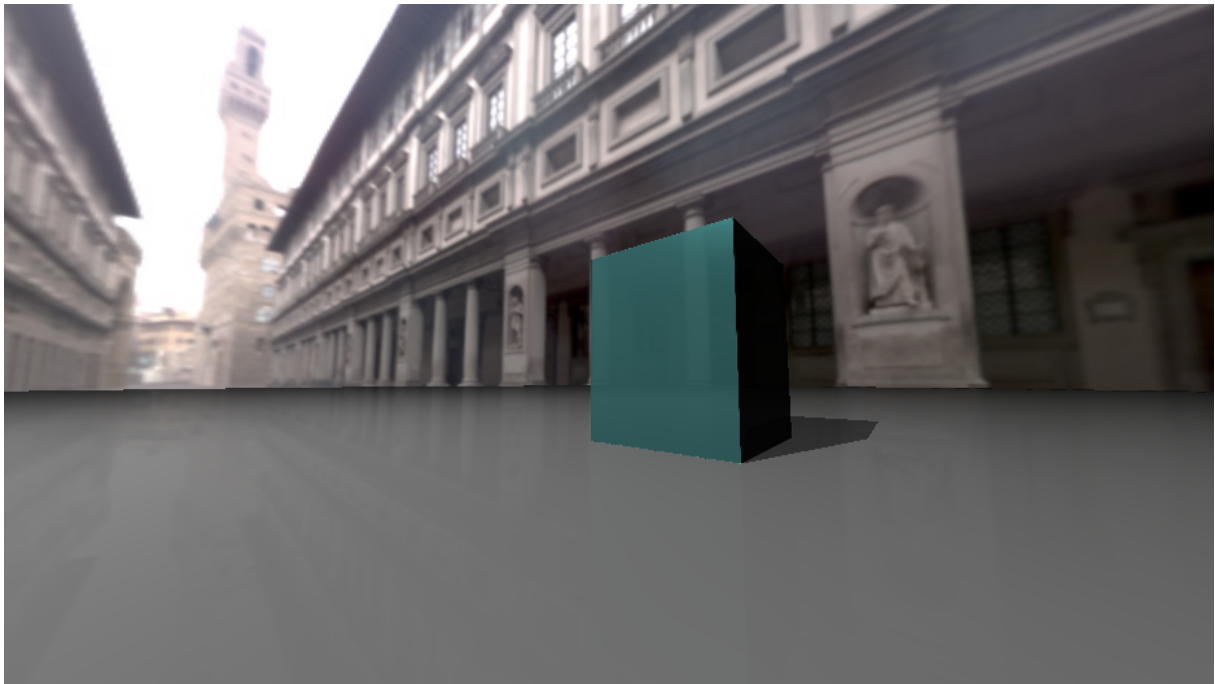


Figure 20: Figure showing a cuboid with true reflections

4.9 Implementation of the Fresnel term

An additional contribution to generating a more realistic shader is adding the Fresnel term. This can be done via the following line:

```
float fresnel = pow(clamp(1.-dot(n, -rd), 0., 1.), 5.);
```

This is GLSL code for what we derived in section 3.5.3.

To give an object the contribution, we use the *mix* function with the *refFact* variable we defined in the previous section.

```
refFact = vec3(mix(.01, .2, fresnel));
```

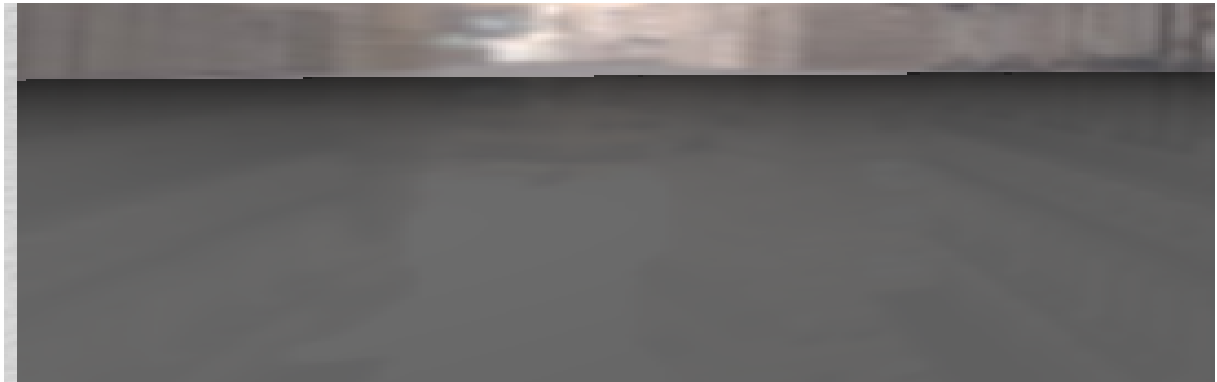


Figure 21: Figure showing a plane with Fresnel contribution

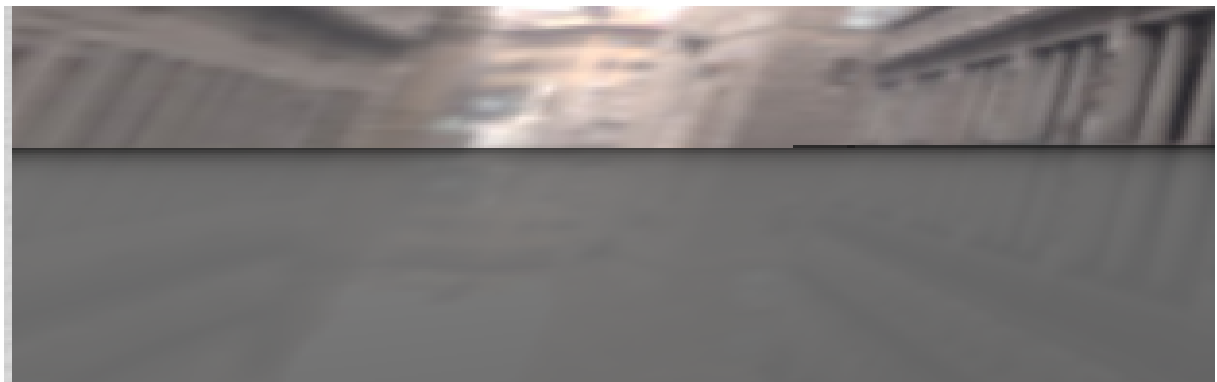


Figure 22: Figure showing a plane with Fresnel contribution

Unfortunately, this effect is difficult to show through images alone. The images above are zoomed in so the pixels are more visible at the cost of the image resolution, yet the effect is still negligible

4.10 The final picture

Now that we have everything we need to construct the shader, we can combine all the techniques. The code for the shader has been included in the appendix (chapter 6), and below are screenshots of the functioning shader.



Figure 23: Final shader from afar, camera from the right

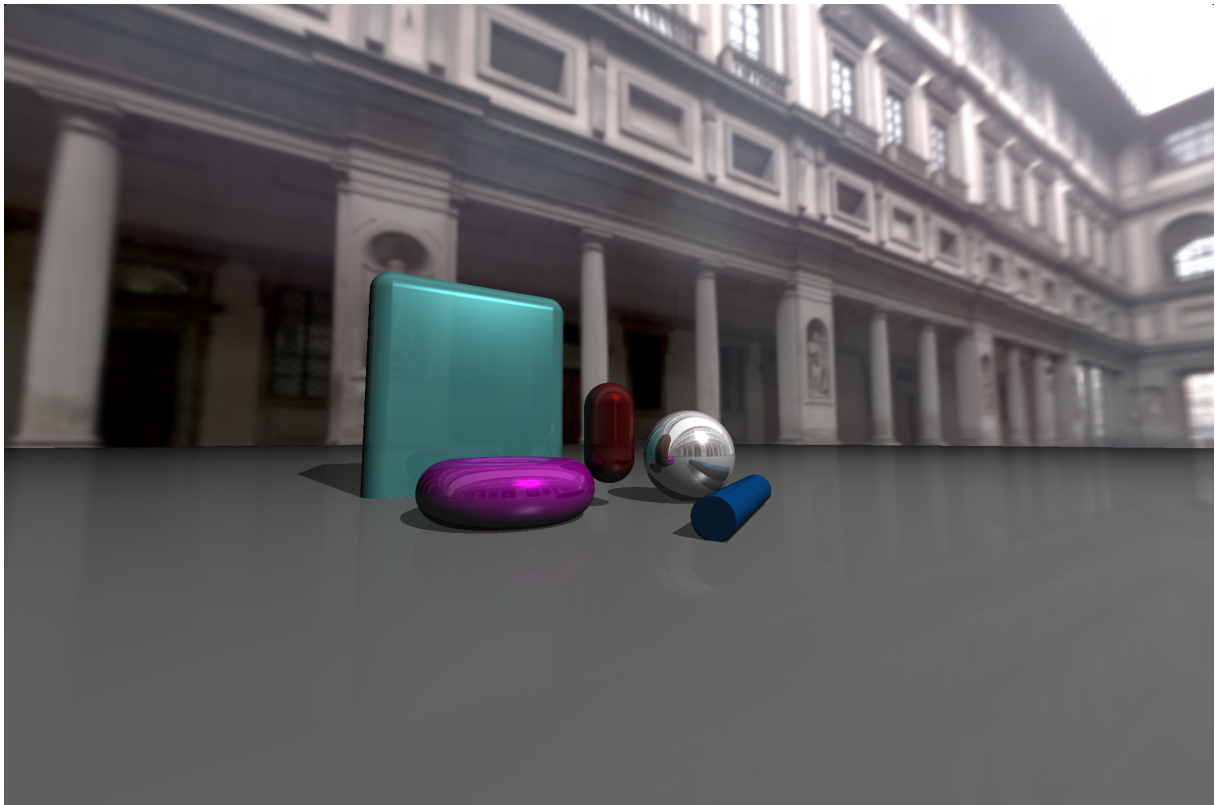


Figure 24: Final shader from afar, camera from the left

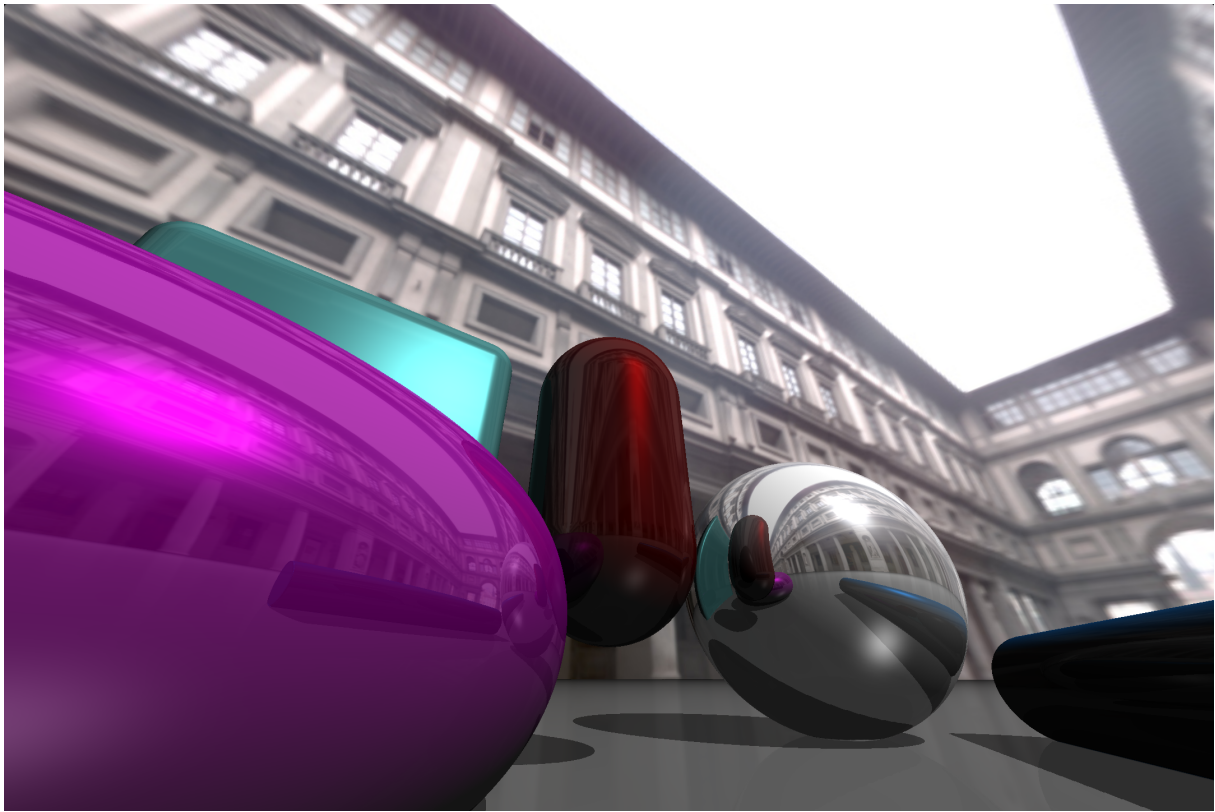


Figure 25: Final shader close-up 1

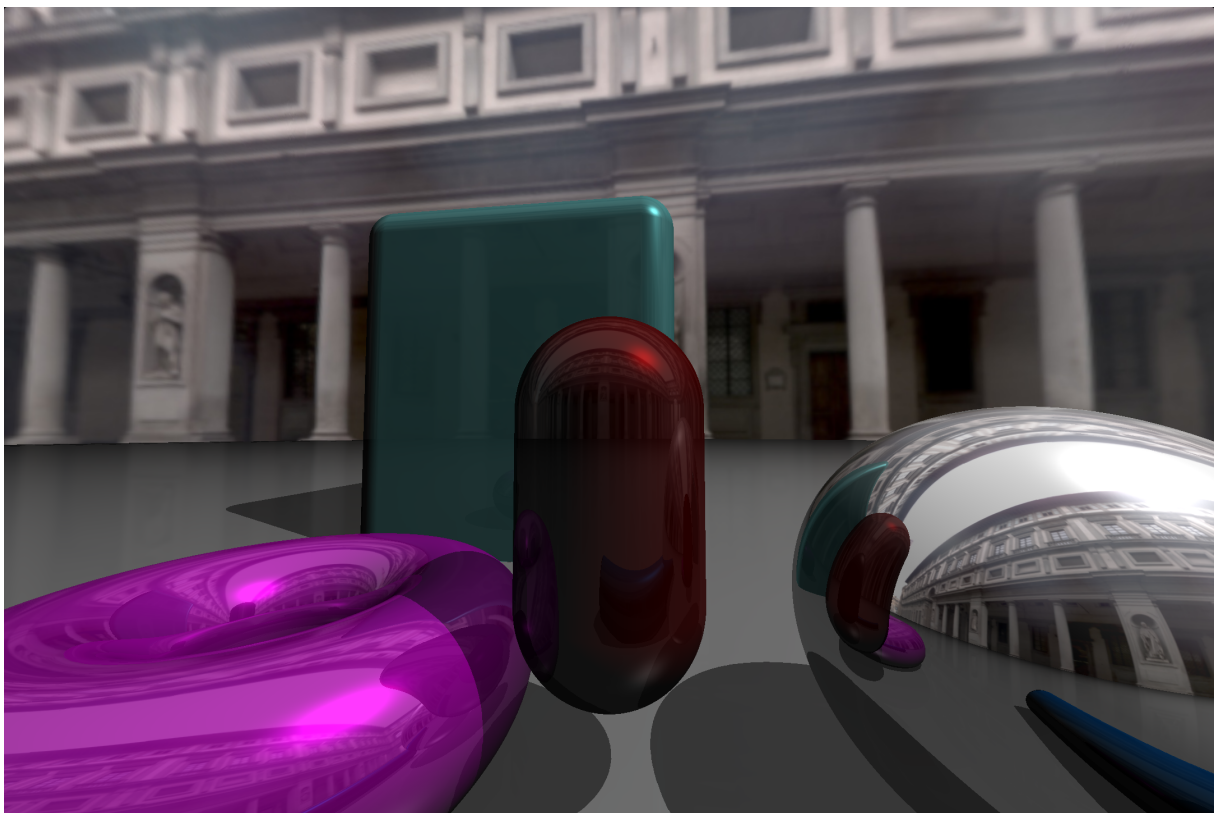


Figure 26: Final shader close-up 2

5 Conclusion

5.1 Summary

The story so far that's been established is the history of ray tracing and how those ideas have evolved into ray marching. We've covered how ray marching came to be, the idea behind sphere tracing, and the mathematics of rendering the geometry and graphical techniques within the three-dimensional space.

The goal of the project was to introduce and implement sphere tracing as a method of rendering virtual objects onto the computer screen. The key piece of theory for implementing this was that of signed distant functions. These, as explained in section 2, are functions that define the geometry in space by returning the smallest distance to the object from a point. Understanding how these functions work is crucial to be able to construct and manipulate far more complex geometry.

In addition to understanding SDFs, there lies an importance to understanding at least the basic lighting models that have been discussed in this paper. Particular importance falls on the Lambertian lighting model which uses the central finite difference method to calculate surface normals, which in turn is used to calculate the diffused lighting for all the objects in the scene. Without the diffused light, the objects will not have shadows and will not be aesthetically pleasing for the viewer.

These functions and techniques are easily transferable to my language of choice for this project, GLSL. GLSL has been incredibly resourceful as a programming language for showcasing the capabilities of ray marching as a technique in computer graphics. Each of the SDFs were able to be more than adequately replicated as a function in the language, and the additional lighting and shading techniques for rendering realistic geometry in GLSL gave very visually pleasing results.

5.2 Limitations

The most broad limitation that hindered the scope of this project was time. With more time, I believe a more complex scene with stronger geometry and more realistic lighting techniques would be possible. Beyond the time limitations, this section will seek to critique and analyse the rest of the shortcomings and limitations of the paper.

5.2.1 Chapter 1: Introduction

Ray marching, and ray tracing, are concepts which are currently decades old. It's almost been a century since computer graphics was created. A recommendation for improvement regarding chapter 1 would be for a more thorough investigation into how similar rendering techniques came to be, and drawing a comparison between the efficiencies of these methods.

Additionally, a vector representation of solving the quadratic would have been a quicker solution than solving the parametric equations.

In a larger project, it would be possible to replicate these findings with sphere tracing and repeat with another rendering technique, potentially multiple, and compare outcomes in terms of efficiency and accuracy.

5.2.2 Chapter 2: Signed distant functions with generating relevant geometry

An initial call for improvement regarding chapter 2 would be to introduce more complex SDFs into the report. In Hart's paper, he created a directory of possible signed distance functions and bounds for a number of primitives [3][pg.4] such as cones, ellipsoids and even fractals, and many more.

In terms of writing the paper and showing visual representations of the SDFs, all the figures that were presented were hand drawn within Goodnotes, an iOS application. Unfortunately, sophisticated vector drawing applications require either a payment and/or a lot of time to learn the software, which considering a lot of time was spent in learning a new programming language, meant I had to sacrifice the use of vector imagery for a more stylised representation for figures in this paper.

5.2.3 Chapter 3: Illumination and shading theory

With computer graphics being such a long studied field, there are many techniques I have omitted from discussing regarding the lighting, shadows or materials of the rendered objects.

One piece of the puzzle which I attempted but had to exclude due to its incompleteness and lack of functionality was transparency. Being able to have certain objects transparent or have a level of transparency would have been a good technique to showcase within GLSL, and it is completely possible as made apparent by the many available shaders on shadertoy which show transparency.

In addition to transparency, a more complex approach to drawing shadows could have been undertaken. At the moment, the shadows generated within my shader are very simple. Soft shadows [10] could have been introduced into my shader given more time. These are methods of diffusing the shadows around the edges of the shadow and lightening super dark areas of the shadows to give a far more realistic shadow. It is rare that a shadow gives pitch blackness in real life, and this sets out to replicate that effect. Soft shadows are prevalent in many video games today.

5.2.4 Chapter 4: Implementation with GLSL

The main limitation regarding GLSL was simply my understanding of the language. Having only started learning the language alongside writing this report, I am still very much a rookie in terms fully utilising the capabilities of the language. One metric for the quality of my code are the performance considerations when running the code, as unnecessarily taxing code will reduce the frame rate of the shader and heat up the GPU. An example of this is repeated code.

5.3 Appraisals

Despite the shortcomings, I am in belief that the results of this report are largely successful. The explanations for signed distance functions and the Lambertian light model are the particular mathematical highlights in the paper, and learning a new language has its own merits. Especially as the language has similar syntax to C, meaning that the skills learnt here will be transferable if I were to move onto programming with C.

5.4 Further study

Given the hypothetical extra time, identifying more key visual improvements to my shader would be my first priority. Mainly to further this project, I would recommend a shader to be constructed featuring a scene with real objects and not just shapes, for example a Newton's cradle. The YouTube channel *Art of code* modelled this and was a great inspiration to this project.

One feature of GLSL I did not explore was its ability to perform animations. The Newton's cradle mentioned above is an example of a shader which would benefit from animations.

Additionally, I briefly mentioned that for a more comprehensive project, I would draw comparisons between sphere tracing and other rendering techniques. One of these rendering techniques is bisection ray marching [6]. This method marches along a ray at fixed intervals until it oversteps the intersection. Once this condition is met, the march backtracks to the midpoint of the last two intervals, and performs a bisection algorithm. This is repeated until the distance to the intersection is so small that we consider it to be an intersection, similar to how we consider an intersection for sphere tracing. It would be interesting to see the performance differences between working with a sphere tracing algorithm compared to a bisection ray marching algorithm.

6 Appendix

```
#define MAX_STEPS 100

#define MAX_DIST 100.

#define SURF_DIST .01

#define NUM_OF_REFLECTION 2

//defining variables


const int MAT_cuboid=1;

const int MAT_torus=2;

const int MAT_sphere=3;

const int MAT_capsule=4;

const int MAT_cylinder=5;

const int MAT_plane=6;


mat2 rotation (float a)

//function to return a matrix which is rotating

{

    float s=sin(a), c=cos(a);

    return mat2(c, -s, s, c);

}


float sdCapsule (vec3 p, vec3 a, vec3 b, float r)

{

    vec3 ab = b-a;

    vec3 ap = p-a;

    float t = dot(ab, ap) / dot(ab, ab);
```

```

    t = clamp(t,0.,1.);

    vec3 c = a + t*ab;

    return length(p-c) - r;
}

```

```

float sdTorus (vec3 p, vec2 r)
{
    float x = length(p.xz)-r.x;

    return length(vec2(x,p.y))-r.y;
}

```

```

float sdBox(vec3 p, vec3 b)
{
    vec3 q = abs(p) - b;

    return length(max(q, 0.0)) + min(max(q.x, max(q.y, q.z)), 0.0)-.5;

    //return length(max(abs(p)-b,0.));
}

```

```

float sdCylinder (vec3 p, vec3 a, vec3 b, float r)
{
    vec3 ab = b-a;

    vec3 ap = p-a;

    float t = dot(ab, ap) / dot(ab, ab);

    //t = clamp(t,0.,1.);

    vec3 c = a + t*ab;

```

```

float x = length(p-c) - r;

float y = (abs(t-.5)-.5)*length(ab);

float e = length(max(vec2(x,y),0.));

float i = min(max(x,y), 0.);

//gives performance penalty, interior distance to improve step estimations

return e+i;
}

```

```

vec2 GetDist(vec3 p)

//the function which displays the objects onto the scene

//combined with the function to get the material of the object

//calculating distance to a plane
{
    vec4 s = vec4(2, 1, 6, 1);

    //define sphere, position xyz and radius r

    //Basic operators

    float plane = p.y;

    vec3 rp = p-vec3(0,1,0);

    rp.xz *= rotation(iTime);

    float sphereDist = length(p-s.xyz)-s.w;

    //calculating distance to the sphere

    float capDist = sdCapsule(p, vec3(0 ,1 ,6), vec3(0, 2, 6), 0.5);

```

```

//calculating distance to the capsule

float torDist = sdTorus(p-vec3(-2,.5, 6), vec2(1.,.5));

//calculating distance to the torus

float boxDist = sdBox(p-vec3(-2,1,12), vec3(2,4,2));

//calculating distance to the box

float cylDist = sdCylinder(p, vec3(0, .3, 3), vec3(3, .3, 5), .3);

//calculating distance to the cylinder

float planeDist = p.y;

//calculating distance to the plane

float sphere = min(sphereDist, planeDist);

float d;

d = min(capDist, planeDist);

d = min(d, torDist);

d = min(d, sphereDist);

d = min(d, boxDist);

d = min(d, cylDist);

//return the minumum distance to the hit surface

int mat = 0;

if (d==boxDist)

mat = MAT_cuboid;

```

```

    else if (d==torDist)

mat = MAT_torus;

    else if (d==sphereDist)

mat = MAT_sphere;

    else if (d==capDist)

mat = MAT_capsule;

    else if (d==cylDist)

mat = MAT_cylinder;

    else if (d==planeDist)

mat = MAT_plane;


    return vec2(d, mat);
}

vec2 SphereTrace(vec3 ro, vec3 rd)

//Sphere tracing function
{

    float d0 = 0.;

    //define distance travelled from origin

vec2 dSMat = vec2(0);

    for(int i=0; i<MAX_STEPS; i++)

        //for an integer i, while i is less than the maximum steps (defined), and
            increment i

    {

        vec3 p = ro + rd*d0;

```

```

        //calculate point p, which is the distance travelled in that single
        marching step

        dSMat = GetDist(p);

        //calculate distance to scene with GetDist function

        d0 += dSMat.x;

        //redefine distance from the origin by adding the distance to the scene

        if(d0>MAX_DIST || dSMat.x<SURF_DIST) break;

        //leave loop if we reach some maximum distance (no hit) or if the
        distance is so small we can classify as a hit

    }

    return vec2(d0, dSMat.y);
}

vec3 GetNormal(vec3 p)
{
    float d = GetDist(p).x;

    //calculate the distance at point p

    vec2 e = vec2(.01, 0);

    vec3 n = d - vec3(
        GetDist(p-e.xyy).x,
        GetDist(p-e.yxy).x,
        GetDist(p-e.yyx).x);

    //making a swizzle ??? of line above????

    //evaluate the distance to the surface around p

    return normalize(n);
}

float GetLight (vec3 p, vec3 viewer)

```

```

{

    vec3 lightPos = vec3(5, 20, -5);

    //position of light source


    lightPos.xz += vec2(sin(iTime), cos(iTime))*10.;

    //moving the light source around the object


    vec3 l = normalize(lightPos-p);

    vec3 n = GetNormal(p);

    //creating the light and normal vectors to calculate the shading dot product


    float dif = clamp(dot(n, l), 0., 1.);

    //Calculate the dot product to know the level of shading.

    //Clamped between 0 and 1 as -1 is a potential result which causes issues
        on complex materials


    vec3 refl = 2.*dot(n, l)*n - l;


    vec2 d = SphereTrace(p+n*SURF_DIST*2., l);

    //Ray March from the bottom of the shadow to the light source

    //but moving the point away from the surface a little bit prevents the ray
        march from ending right away

    if(d.x<length(lightPos-p)) dif *= .5;

    //if the distance is less than the distance between the light and the
        surface position

    //then we know that we are drawing the shadow

    //the brightness of the shadow denoted by the dif *=.1

    return .5*dif + 0.75*pow(clamp(dot(refl, -viewer), 0., 1.), 30.0);
}

```

```

    // phong
}

vec3 Render(inout vec3 ro, inout vec3 rd, inout vec3 refFact)
{
    vec3 col = texture(iChannel0, rd).rgb;

    //background

    vec2 dMat = SphereTrace(ro,rd);

    vec3 p = ro + rd * dMat.x;

    //calculate point p in the 3d space after the ray march

    vec3 n = GetNormal(p);

    vec3 r = reflect(rd, n);

    p += n*0.001;

    //vec3 ref = texture(iChannel0, r).rgb;

    //fresnel term

    float fresnel = pow(clamp(1.-dot(n, -rd), 0., 1.), 5.);

    //requires camera to be working the way I want

    refFact = vec3(0);

    //ensuring that if we miss an object, then there are no reflections

    if(dMat.x<MAX_DIST)
    {
        float dif = GetLight(p, rd);

        col = vec3(dif);
    }
}

```

```

int mat = int(dMat.y); //GetMaterial(p);

if(mat==MAT_cuboid)
{
    col *= vec3(0.5,1,1);
    refFact = vec3(.1);
}

else if (mat==MAT_sphere)
{
    //col *= ref; //vec3(0,0,1);
    refFact = vec3(.75);
}

else if (mat==MAT_torus)
{
    col *= vec3(1,0,1);
    refFact = vec3(.3);
}

else if (mat==MAT_capsule)
{
    col *= vec3(.5,0,0);
    refFact = vec3(.2);
}

else if (mat==MAT_cylinder)
{
    col *= vec3(0,.5,1);
    refFact = vec3(.1);
}

```

```

    else if (mat==MAT_plane)
    {
        refFact = vec3(mix(.01, .2, fresnel));
    }
}

ro = p+n*SURF_DIST;

rd = r;

return col;
}

vec3 defrd(vec2 uv, vec3 p, vec3 l, float z)
// l is the rotation point in space
{
    vec3 f = normalize(l-p), //normalise distance from l to p
    r = normalize(cross(vec3(0,1,0), f)), //calculate cross product of vectors
        f and (0,1,0)
    u = cross(f,r), //cross product again
    c = f*z, //zoom in z direction
    i = normalize(c + uv.x*r + uv.y*u); //formulating ray direction
    return i;
}

void mainImage( out vec4 fragColor, in vec2 fragCoord )
{
    vec2 uv = (fragCoord-.5*iResolution.xy)/iResolution.y;

    //Normalised uv with 0 uv in the middle of the scree

    vec2 m = iMouse.xy/iResolution.xy;

```

```

vec3 ro = vec3(0, 3, -3);

ro.yz *= rotation(-m.y*3.14+1.);

ro.xz *= rotation(-m.x*6.2831);

ro.y = max(ro.y, .1);

//ensuring camera does not go below the plane


// vec3 ro = vec3(0, 3, -3);

//Define ray origin at point (0,1,0)


vec3 rd = defrd(uv, ro, vec3(0, 2, 6), .5);

//Define ray direction through the function


vec3 refFact = vec3(0);

vec3 filt = vec3(1);

vec3 col = Render(ro, rd, refFact);

//background and colour to objects


//const int numOfRef = 2;


for(int i=0; i<NUM_OF_REFLECTION; i++)
{
    filt *= refFact;

    vec3 reflected = filt*Render(ro, rd, refFact);

    col += reflected*.75;
}


//float dif = GetLight(p);

```

```

//Getting the diffused lighting with the GetLight function

//col = vec3(dif);

fragColor = vec4(col,1.0);
}

```

7 Bibliography

References

- [1] History of Computer Graphics, <https://www.tutorialandexample.com/history-of-computer-graphics>, (Accessed 01/11/2021)
- [2] Foley, *Computer Graphics: Principles and Practice Second Edition*. 1990
- [3] J.C. Hart, *Sphere tracing: a geometric method for the antialiased ray tracing of implicit surfaces*. 1997
- [4] MÖLLER, T., AND TRUMBORE, *Fast, minimum storage ray/triangle intersection*. 1997
- [5] Rendering Implicit Surfaces and Distance Fields: Sphere Tracing, <https://www.scratchapixel.com/lessons/advanced-rendering/rendering-distance-fields/maths-behind-sphere-tracing>, (Accessed 03/11/2021)
- [6] Rendering Implicit Surfaces and Distance Fields: Sphere Tracing, <https://www.scratchapixel.com/lessons/advanced-rendering/rendering-distance-fields> (Accessed 09/03/2022)
- [7] Art of code, Ray marching simple shapes, <https://youtu.be/Ff0jJyyiVyw> Accessed 02/01/2022
- [8] Art of code, Ray marched reflections, <https://youtu.be/beNDx5Cvt7M> Accessed 20/01/2022
- [9] Wikipedia contributors, OpenGL Shader Language, https://en.wikipedia.org/wiki/OpenGL_Shading_Language (Accessed 10/02/2022)
- [10] Wikipedia contributors, Hard and Soft Light, https://en.wikipedia.org/wiki/Hard_and_soft_light (Accessed 12/03/2022)
- [11] Inigo Quilez, normals for an SDF, <https://www.iquilezles.org/www/articles/normalsSDF/normalsSDF.htm> (Accessed 1/03/2022)